# Embedded Application Development

## Application Note

**SUMMARY:** This document describes the application development software architecture, which provides a framework for customers to develop their Bluetooth LE applications and products based on the Atmosic Wireless SoC Series.

CONFIDENTIAL



Atmosic™

# Table of Contents

**1. Overview** 6

1.1 Software Architecture 6

1.2 Application Framework 7

**2. Bluetooth LE Application** 14

2.1 Bluetooth LE Connection Flow 14

2.1.1 Atmosic GAP Modules 15

atm_gap_param Module 15

atm_gap Module 18

2.2 Profile Registration 19

2.3 Initialization 20

2.4 Device Discovery - Advertising 23

2.4.1 Atmosic Advertisement Modules 23

2.4.2 atm_adv_param Module 24

Get parameter from NVDS 25

Get parameter from predefined structure 26

Using runtime modified parameter 27

2.4.3 atm_adv Module 28

2.5 Device Discovery -Scanning 30

2.5.1 Atmosic Scan Modules 31

2.5.2 atm_scan_param Module 31

Get parameter from NVDS 32

Get parameter from predefined structure 32

Using runtime modified parameter 33

2.5.3 atm_scan Module 34

2.6 Connection Establishment 37

2.6.1 Atmosic Initiator Modules 37

2.6.2 atm_init_param Module 38

Get parameter from predefined structure 38

Using runtime modified parameter 39

2.6.3 atm_init Module 39

2.7 Connection Mechanism 42

CONFIDENTIAL

2.7.1 Detailed Bluetooth LE Connection Flow 42
**3 System Utilities** 45
3.1 Application State Machine 45
3.2 Log Utility 45
3.2.1 ATM_LOG Macro 45
3.2.2 Debug Log Level 45
3.2.3 How to Use 46
3.3 Power Management 47
3.3.1 Power Saving Modes 47
3.3.2 Power Mode Lock 47
3.3.3 Power Manager API 48
3.3.4 atm_pm lock example 48
3.4 Software Timer 49
3.4.1 Software Timer API 49
3.4.2 sw_timer example 50
3.5 Software Event 51
3.5.1 Software Event API 51
3.5.2 sw_event example 52
3.6 AT Command 53
**4. Hardware Drivers** 54
4.1 GPIO Driver 54
4.1.1 atm_gpio Module 54
4.2 I2C Driver 56
4.2.1 i2c Module 56
4.3 SPI Driver 62
4.3.1 Pin Assignment for SPI 62
4.3.2 SPI Module 63
**Revision History** 66

CONFIDENTIAL

# List of Figures

Figure 1 - Atmosic Application Software Architecture

Figure 2 - Overview of the Atmosic Application Framework Connection Flow

Figure 3 - Profile Registration

Figure 4 - Initialization Process

Figure 5 - Advertisement Modules

Figure 6 - atm_adv_param Flow

Figure 7 - Get Parameter from Predefined Structure

Figure 8 - Using Runtime Modified Parameter

Figure 9 - atm_adv Flow

Figure 10 - Scan Module Usage Diagram

Figure 11 - atm_scan_param Module Diagram

Figure 12 - atm_scan_param Module Pre-defined Structure

Figure 14 - atm_scan Module Flow

Figure 15 - Initiator Modules

Figure 16 - Initiator Parameter Module

Figure 17 - atm_init Flow

Figure 18 - Connection Mechanism Sequence Overview

Figure 19 - Atmosic Application Framework Initialization Sequence

Figure 20 - Atmosic Application Framework Connection Mechanism Sequence

Figure 21 - Power Mode Lock

Figure 22 - atm_gpio Module Flow

Figure 23 - i2c Module Flow

Figure 24- Timing Diagram For SPI Mode 0 - Sample At Rising Edge

Figure 25- SPI Pin

Figure 26 - SPI Module

CONFIDENTIAL

# List of Tables

Table 1 - List of Supported Devices

Table 2 - Atmosic application framework module list

Table 3 - atm_gap_param_t Atmosic GAP parameters

Table 4 - Device Configurations

Table 5 - Frequently Used GAP API Functions

Table 6 - Frequently Used GAP Callbacks

Table 7 - NVDS Parameters

Table 8 - API Parameters Description

Table 9 - Initiator Parameters API Description

Table 10 - Debug Log Level

Table 11 - Power Saving Modes

Table 12 - Power Manager API

Table 13 - Software Timer APIs

Table 14 - Software Event APIs

Table 15 - Atmosic GPIO API Description

Table 16 - Atmosic I2C API Description

Table 17 - Atmosic SPI API Description

CONFIDENTIAL

# 1. Overview

The Atmosic Application software supports the following devices:

Table 1 - List of Supported Devices

| ATM2 Series | ATM3 Series | ATM33 Series |
|---|---|---|
| ATM2201 | ATM3201 | ATM3330e |
| ATM2202 | ATM3202 | ATM3330 |
| ATM2221 | ATM3221 | ATM3325 |
| ATM2251 | ATM3251 | |

CONFIDENTIAL

## 1.1 Software Architecture

The software for Atmosic SoC includes 3 layers:

- Application
- Application framework
- Protocol stack and hardware registers

Figure 1 shows application software architecture.

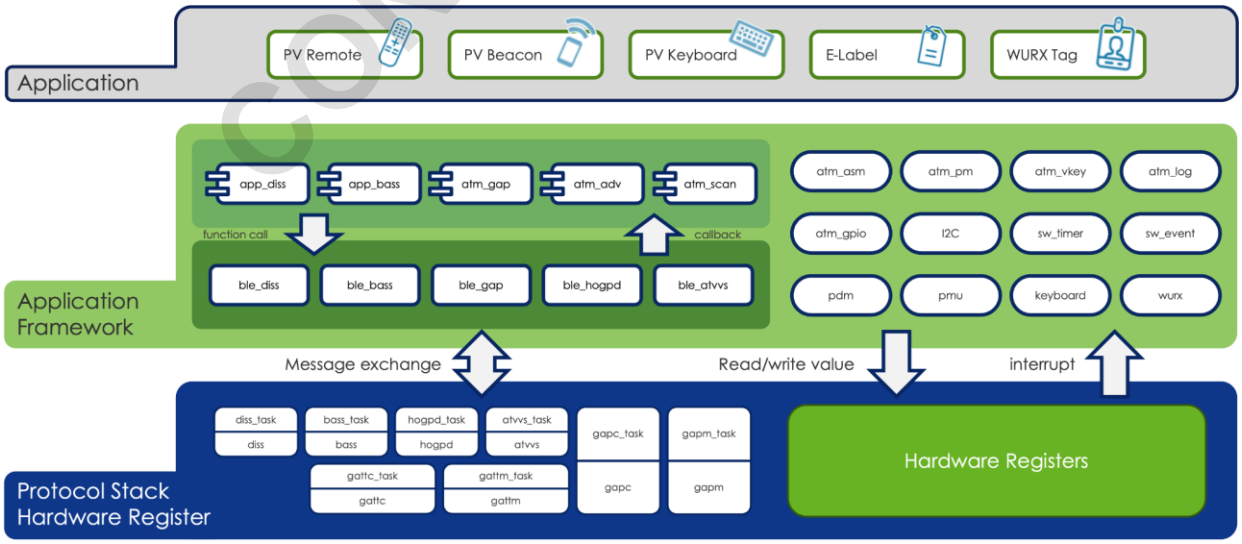



*Figure 1 - Atmosic Application Software Architecture*

The application software is written in C using functions provided by the application framework. Currently some basic projects, such as broadcaster and observer, and

some reference design projects, such as remote controller and Bluetooth LE UART bridge, are provided.

## 1.2 Application Framework

Atmosic Application Framework is a set of C library functions for application development. It was designed and developed based on CEVA RivieraWaves Bluetooth Low Energy software protocol stack and hardware registers.

The purpose of the application framework is to allow user to develop Bluetooth LE applications and products based on Atmosic SoC efficiently.

The application framework consists of application adapters (Bluetooth LE activities and profile managers), Atmosic profiles, stack binders, system utilities and hardware drivers. The stack binder transforms the function calls from application adapters into the sending messages to CEVA stack or the receiving messages from CEVA stack into the callback invokings to application adapters. The application adapter provides high level APIs for applications to interact with the profiles. The driver provides the functionality and configuration APIs to access Atmosic hardware devices.

Table 2 lists all Atmosic application framework modules and their filenames and description.

CONFIDENTIAL

Table 2 - Atmosic application framework module list

| Category | Path | File Name | Description |
|---|---|---|---|
| Application Adapter | lib/app_bass | app_bass.c<br>app_bass.h | Battery Service (BAS) server application adapter |
| | lib/app_batt | app_batt.c<br>app_batt.h | Battery procedure for application layer |
| | lib/app_+diss | app_diss.c<br>app_diss.h | Device Information Service (DIS) server application adapter |
| | lib/app_fio | app_fio.c<br>app_fio.h | Function IO control for application layer |
| | lib/app_gap | app_gap.c<br>app_gap.h | LE Generic Attribute Profile (GAP) application adapter |

| Category | Path | File Name | Description |
|---|---|---|---|
| | lib/app_hrps | app_hrps.c<br>app_hrps.h | Heart Rate Profile (HRP) server application adapter |
| | lib/app_htpt | app_htpt.c<br>app_htpt.h | Heath Thermometer Profile (HTP) server application adapter |
| | lib/app_otaps | app_otaps.c<br>app_otaps.h | Atmosic firmware update Over-The-Air Profile (OTAP) server application adapter |
| | lib/atm_gap | atm_gap.c<br>atm_gap.h<br>atm_gap_param.c<br>atm_gap_param.h<br>atm_gap_param_internal.h | LE device and link manager |
| | lib/atm_adv | atm_adv.c<br>atm_adv.h<br>atm_adv_param.c<br>atm_adv_param.h<br>atm_adv_param_internal.h | LE advertising manager |
| | lib/atm_persync | atm_persync.c<br>atm_persync.h | LE periodic sync. procedures |
| | lib/atm_scan | atm_scan.c<br>atm_scan.h<br>atm_scan_param.c<br>atm_scan_param.h<br>atm_scan_param_internal.h | LE scanning manager |
| | lib/atm_init | atm_init.c<br>atm_init.h<br>atm_init_param.c<br>atm_init_param.h<br>atm_init_param_internal.h | LE Initiating manager |
| Atmosic Profile | lib/prf_bridge | prf_bridge.c<br>prf_bridge.h | Atmosic bridge profile<br>(based on Atmosic profile server) |

CONFIDENTIAL

| Category | Path | File Name | Description |
|---|---|---|---|
| | lib/atm_prfs | atm_prfs.c<br>atm_prfs.h<br>atm_prfs_task.c<br>atm_prfs_task.h | Atmosic profile server |
| Stack Binder | lib/ble_task | ble_task.c<br>ble_task.h | BLE application task message handler |
| | lib/ble_gap | ble_gap.c<br>ble_gap.h | GAP API |
| | lib/ble_gap_sec | ble_gap_sec.c<br>ble_gap_sec.h | GAP security part API |
| | lib/ble_ancsc | ble_ancsc.c<br>ble_ancsc.h | ANCS client API |
| | lib/ble_atvvs | ble_atvvs.c<br>ble_atvvs.h | ATVV API |
| | lib/ble_bass | ble_bass.c<br>ble_bass.h | BAS server API |
| | lib/ble_common | ble_common.h<br>ble_err.h | BLE common and error code definitions |
| | lib/ble_diss | ble_diss.c<br>ble_diss.h | DIS server API |
| | lib/ble_hogpd | ble_hogpd.c<br>ble_hogpd.h | HOGPD API |
| | lib/ble_atvvs | ble_atvvs.c<br>ble_atvvs.h | Android TV voice service (ATVVS, aka Google Voice Service) server API |
| | lib/ble_atmprfs | ble_atmprfs.c<br>ble_atmprfs.h | Atmosic profile server API |

CONFIDENTIAL

| Category | Path | File Name | Description |
|---|---|---|---|
| | lib/ble_gattc | ble_gattc.c<br>ble_gattc.h | GATT message handler worked with ROM profile stack |
| | lib/ble_hrps | ble_hrps.c<br>ble_hrps.h | HRP server API |
| | lib/ble_htpt | ble_htpt.c<br>ble_htpt.h | HTP server API |
| | lib/ble_lecb | ble_lecb.c<br>ble_lecb.h | LE credit based connection procedures |
| | lib/ble_module | ble_module.c<br>ble_module.h | Link time module |
| | lib/ble_otaps | ble_otaps.c.c<br>ble_otaps.h | OTAP server API |
| | lib/ble_tputps | ble_tputps.c<br>ble_tputps.h | TPUTPS profile control API |
| System Utilities | lib/atm_asm | atm_asm.c<br>atm_asm.h | Application state machine API |
| | lib/atm_bootutil | atm_bootutil.c<br>atm_bootutil.h | Boot utilities<br>(**ATM33/e only**) |
| | lib/atm_common | atm_co_bt_defines.h<br>atm_co_utils.h<br>atm_ke_utils.h | Framework common utilities |
| | lib/atm_debug | atm_debug.c<br>atm_debug.h | Debug utilities |
| | lib/atm_log | atm_log.h | Log utility |
| | lib/atm_tinycrypt_port | atm_tinycrypt_prot.h | External config file for tinycrypt<br>(**ATM33/e only**) |

CONFIDENTIAL

| Category | Path | File Name | Description |
|---|---|---|---|
| | lib/atm_utils_x | atm_utils_c.h<br>atm_utils_endian.h<br>atm_utils_math.h | Common utilities |
| | lib/sec_service | sec_service.c<br>sec_service.h | Secure services<br>(**ATM33/e only**) |
| | driver/atm_pm | atm_pm.c<br>atm_pm.h | Power management API |
| | driver/sw_event | sw_event.c<br>sw_event.h | Software event API |
| | driver/sw_timer | sw_timer.c<br>sw_timer.h | Software timer API |
| | lib/at_cmd | at_cmd.c<br>at_cmd.h<br>at_cmd_pasr.c<br>at_cmd_pasr.h | Atmosic AT command engine |
| | lib/at_cmd_set | at_cmd_event.h<br>at_cmd_init.c<br>at_cmd_init.h<br>at_cmd_utils.c<br>at_cmd_utils.h<br>at_cmd_dfu_flash.c<br>at_cmd_dfu_flash.h<br>at_cmd_dfu_proc.c<br>at_cmd_dfu_proc.h<br>at_cmd_rawproto.c<br>at_cmd_rawproto.h<br>[at_cmd_handlers..] | AT command generic API and existing command handlers |
| Hardware Driver | driver/atm_ble | atm_ble.c<br>atm_ble.h | Bluetooth LE stack API extension |
| | driver/atm_button | atm_button.c<br>atm_button.h | Button driver based on GPIO |

CONFIDENTIAL

| Category | Path | File Name | Description |
|---|---|---|---|
| | driver/atm_gpio | atm_gpio.c<br>atm_gpio.h | GPIO driver |
| | driver/atm_vkey | atm_vkey.c<br>atm_vkey.h | Virtual key event model |
| | driver/batt_model | batt_model.c<br>batt_model.h | Battery model common part |
| | driver/batt_model_coin | batt_model_coin.c | Battery model for coin cell |
| | driver/batt_model_hsc | batt_model_hsc.c | Battery model for hybrid super CAP(HSC) |
| | driver/brwnout | brwnout.c<br>brwnout.h | Brownout support |
| | driver/ext_flash | ext_flash.c<br>ext_flash.h | External flash driver |
| | driver/gadc | gadc.c<br>gadc.h | ADC driver |
| | driver/hib_storage | hib_storage.c<br>hib_storage.h | Hibernation storage driver |
| | driver/hw_cfg | hw_cfg.c<br>hw_cfg.h | Hardware configuration |
| | driver/i2c | i2c.c<br>i2c.h | I2C driver |
| | driver/interrupt | interrupt.c<br>interrupt.h | Interrupt routing and handler |
| | driver/ir | ir.c<br>ir.h<br>nec_ir.c | IR driver |

CONFIDENTIAL

| Category | Path | File Name | Description |
|---|---|---|---|
| | driver/irrx | irrx.c<br>irrx.h | IR receiver driver |
| | driver/keyboard | keyboard.c<br>keyboard.h<br>keyboard_internal.h<br>usb_hid_keys.h | Key scan matrix driver |
| | driver/keyboard_ghostkey | keyboard_ghostkey.c<br>keyboard_ghostkey.h | Keyboard ghost key filter |
| | driver/keyboard_hid | keyboard_hid.c<br>keyboard_hid.h | Keyboard hid driver |
| | driver/led_blink | led_blink.c<br>led_blink.h | LED driver |
| | driver/pdm | adpcm_enc.c<br>adpcm_enc.h<br>pdm_intp_data.h<br>pdm_intp.c<br>pdm_intp.h<br>pdm.c<br>pdm.h | PDM driver and ADPCM encoder |
| | driver/pmu | pmu.c<br>pmu.h | Energy harvesting management API |
| | driver/spi | qspi.h<br>spi_flash.c<br>spi_flash.h<br>spi.c<br>spi.h | SPI/QSPI driver |
| | driver/uart_flash | uart_flash.c | UART flash driver<br>**(ATM2/ATM3 only)** |
| | driver/uart0_raw | uart0_raw.c<br>uart0_raw.h | UART driver |

CONFIDENTIAL

| Category | Path | File Name | Description |
|---|---|---|---|
| | driver/wurx | wurx_rom.c<br>wurx.c<br>wurx.h | Wakeup receiver driver<br>**(ATM2/ATM3 only)** |

# 2. Bluetooth LE Application

## 2.1 Bluetooth LE Connection Flow

Figure 2 depicts the overview of the connection flow to establish a Bluetooth LE connection via Atmosic application framework:

- Profile Registration
- Initialization
- Device discovery
  - Advertising
  - Scanning
- Connection establishment
- Connection mechanism
- Connection detachment

CONFIDENTIAL

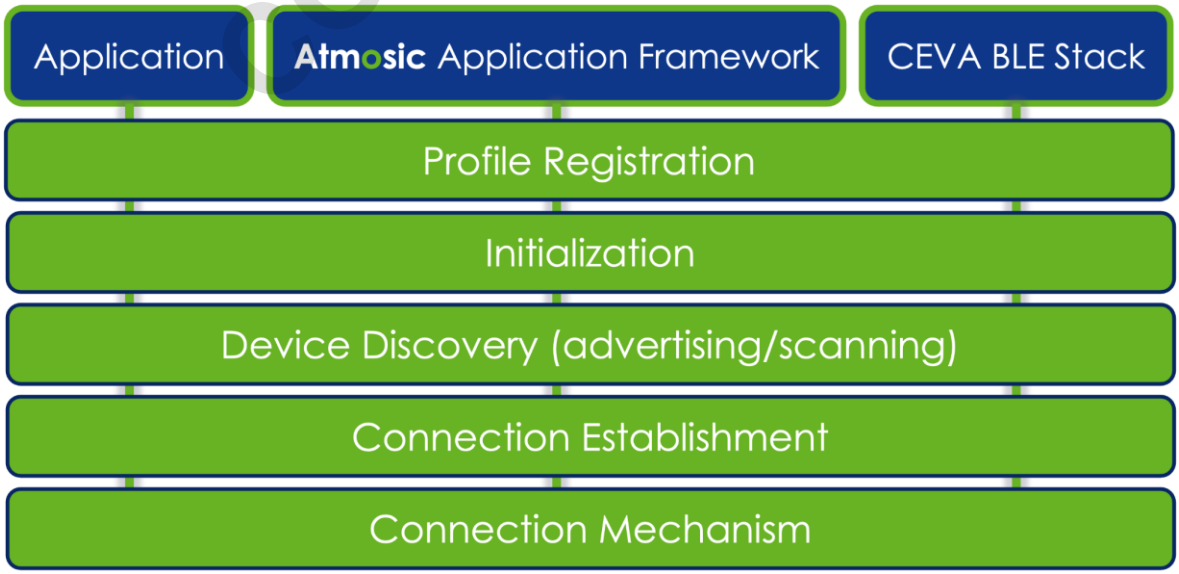



*Figure 2 - Overview of the Atmosic Application Framework Connection Flow*

### 2.1.1 Atmosic GAP Modules

Atmosic GAP modules are the basic modules of Atmosic Application Framework and required for all Bluetooth LE applications. With **atm_gap_param** and **atm_gap** modules, applications could use their API to complete the BLE stack initialization and GAP messages exchange.

#### atm_gap_param Module

The **atm_gap_param** module provides an API to get a set of default parameters for GAP initialization and the header file atm_gap_param_internal defines the default values of GAP configurations in case these configurations are not overwritten in applications. **atm_gap_param_t** is the structure of GAP configuration. All parameters in **atm_gap_param_t** are shown in Table 3.

CONFIDENTIAL

Table 3 - atm_gap_param_t Atmosic GAP parameters

| Member | Description |
|---|---|
| uint8_t* dev_name | dev_name is a pointer to a string array. This string will be the **Device Name (0x2A00)** characteristic value of GAP services . If NVDS tag 0x02 exists, the dev_name will be replaced. |
| uint8_t dev_name_len | Length of device name |
| uint8_t dev_name_max | Size of the device name array |
| uint16_t appearance | **Appearance (0x2A01)** characteristic value of GAP services. |
| **[ATM33/e]**<br>struct gap_periph_pref slv_pref_params<br><br>**[ATM2/ATM3]**<br>struct gap_slv_pref slv_pref_params | **Peripheral(Slave) preferred connection parameters (0x2A04)** characteristic values of GAP services. This configuration includes connection interval minimum, connection interval maximum, slave latency and connection timeout parameters. |
| uint8_t const *app_irk | app_irk is a pointer to Identity Resolving Key (IRK). IRK assignment from application. Null if not used. |
| struct gapm_set_dev_config_cmd dev_config | See Table 3 |
| bd_addr_t addr | Bluetooth device address |

[dev_config] (struct gapm_set_dev_config_cmd dev_config) is the most important configuration in atm_gap_param_t for the initialization procedure to decide the device role, privacy, security and data exchange features. Configurations in dev_config are shown in Table 4.

The definition of structure “gapm_set_dev_config_cmd”:

- ATM2/ ATM3

  with all members

- ATM33/e

  compress all members except for "operation" into the structure "gapm_config_t".

Table 4 - Device Configurations

| Member | Descriptions |
|---|---|
| **Generic Configuration** | |
| uint8_t operation | GAPM requested operation. |
| uint8_t role | Bluetooth LE device role. It would be central, peripheral, observer, broadcaster or all roles. |
| **Privacy Configuration** | |
| uint16_t renew_dur | Address renew duration when controller privacy is enabled. Unit is second. |
| bd_addr_t addr | Device static private random address. If the NVDS tag 0x01 exists, it will be replaced. |
| struct gap_sec_key irk | Device IRK used for resolvable random BD address generation (LSB first). |
| uint8_t privacy_cfg | Privacy configuration. It is used to enable controller privacy. |
| **Security Configuration** | |
| uint8_t pairing_mode | pairing mode is used to enable pairing feature, legacy or SC. |
| **ATT Database Configuration** | |
| uint16_t gap_start_hdl | GAP service start handle. |
| uint16_t gatt_start_hdl | GATT service start handle. |

CONFIDENTIAL

| | |
|---|---|
| uint16_t att_cfg | Attribute database configuration. |
| **LE Data Length Extension Configuration** | |
| uint16_t sugg_max_tx_octets | The Controller's maximum transmitted number of payload octets to be use |
| uint16_t sugg_max_tx_time | The Controller's maximum packet transmission time to be used. |
| **L2CAP Configuration** | |
| uint16_t max_mtu | Maximum MTU acceptable for the device.<br>**(ATM2/ATM3 only)** |
| uint16_t max_mps | Maximal MPS Packet size acceptable for device.<br>**(ATM2/ATM3 only)** |
| uint8_t max_nb_lecb | Maximum number of Bluetooth LE Credit based connections that can be established.<br>**(ATM2/ATM3 only)** |
| **Bluetooth LE Audio Mode Supported** | |
| uint16_t audio_cfg | Bluetooth LE Audio Mode Configuration.<br>**(ATM2/ATM3 only)** |
| **Bluetooth LE PHY Management** | |
| uint8_t tx_pref_phy | Preferred Bluetooth LE PHY for data transmission. |
| uint8_t rx_pref_phy | Preferred Bluetooth LE PHY for data reception. |
| **Miscellaneous 2/ Radio Configuration** | |
| uint16_t tx_path_comp | RF TX Path Compensation value (from -128 dB to 128 dB, unit is 0.1 dB). |
| uint16_t rx_path_comp | RF RX Path Compensation value (from -128 dB to 128 dB, unit is 0.1 dB). |
| **BT Classic Configuration** | |
| uint32_t class_of_device | Bluetooth Class of device.<br>**(ATM33/e only)** |
| uint16_t dflt_link_policy | Default link policy.<br>**(ATM33/e only)** |
| bool ssp_enable | True if Secure simple pairing enabled, False otherwise.<br>**(ATM33/e only)** |

CONFIDENTIAL

## atm_gap Module

The **atm_gap** module provides the required APIs to configure the device and initialize the Bluetooth LE stack with variant configurations according to the application requirements. During the connection mechanism process, it provides APIs to accept the connection request, negotiate the connection parameters and detach the existing links . These API functions are shown in Table 5.

Table 5 - Frequently Used GAP API Functions

| Function | Descriptions |
|---|---|
| **Initialization** | |
| ***atm_gap_prf_reg***(char const *name, void const *parm) | Register profile with profile name and parameters. |
| ***atm_gap_start***(atm_gap_param_t *init, atm_gap_cbs_t const *cbs) | Initialize BBluetooth LE stack with init parameters and callbacks |
| **Connection** | |
| ***atm_gap_connect_accept***(uint8_t conidx) | Accept connection request by connection index |
| ***atm_gap_connect_param_nego***(uint8_t conidx, atm_gap_param_nego_t const *param) | Launch process of requesting new connection parameter |
| ***atm_gap_print_conn_param***(atm_connect_info_t *info) | Print connection parameter |
| ***atm_gap_disconnect***(uint8_t conidx, uint8_t reason) | Disconnect connection with reason by connection index |

CONFIDENTIAL

The **atm_gap** also provides a set of callback functions, **atm_gap_cbs_t** to handle the GAP messages from the BBluetooth LE stack in the applications. The frequently used callback functions are shown in Table 6.

Table 6 - Frequently Used GAP Callbacks

| Callback | Descriptions |
|---|---|
| **Initialization** | |
| ***init_cfm*** | Confirmation of finish of atm_gap_start. |
| ***quick_start_op*** | Quick start operation. Called after bt reset but before |

| | |
|---|---|
| | atm_gap_start finished. |
| **Scan** | |
| ***ext_adv_ind*** | Indicate reception of advertising, scan response or periodic advertising data. |
| **Connection** | |
| ***conn_ind*** | Indicate that a connection has been established. |
| ***disc_ind*** | Indicate that a link has been disconnected |
| ***conn_param_updated_ind*** | Indication that connection parameters have been updated. |
| **Pairing** | |
| ***pair_req_ind*** | Indicate received pairing request from master. |
| ***sec_req_ind*** | Indicate received a security request from the slave. |
| ***pair_passkey_ind*** | Indication of passkey display or passkey input. |
| ***pair_numeric_ind*** | Indication of numeric comparison reception. |
| ***pair_ind*** | Indication of pairing result. |

CONFIDENTIAL

## 2.2 Profile Registration

Profile registration phase is about registering profile API into the **atm_gap** module. After the profile is registered, the events of the registered profile could be received through its callback functions. Application calls **atm_gap_prf_reg** function with profile name and configuration parameters, as shown in Figure 3.

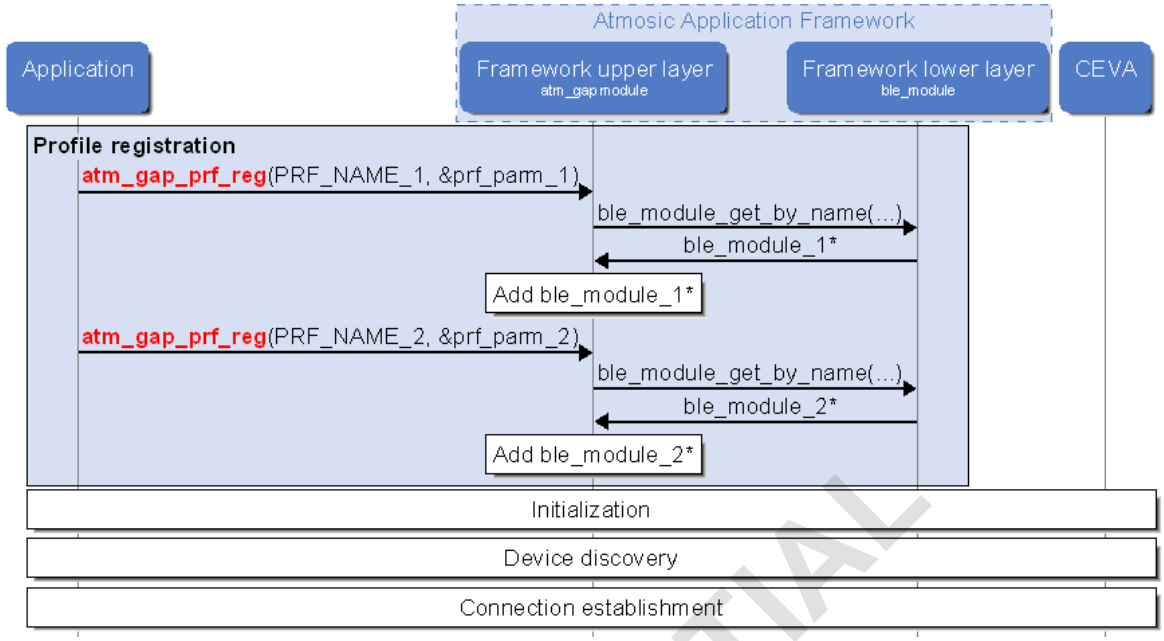



*Figure 3 - Profile Registration*

## 2.3 Initialization

After profile registration, the application calls the atm_gap_start with gap parameter and application callbacks. Once initialization is done, the callback init_cfm which is provided in atm_gap_start argument would be invoked to notify the application, as shown in Figure 4. Refer to the Detailed Bluetooth LE Connection Flow section for details of this initialization sequence.

CONFIDENTIAL

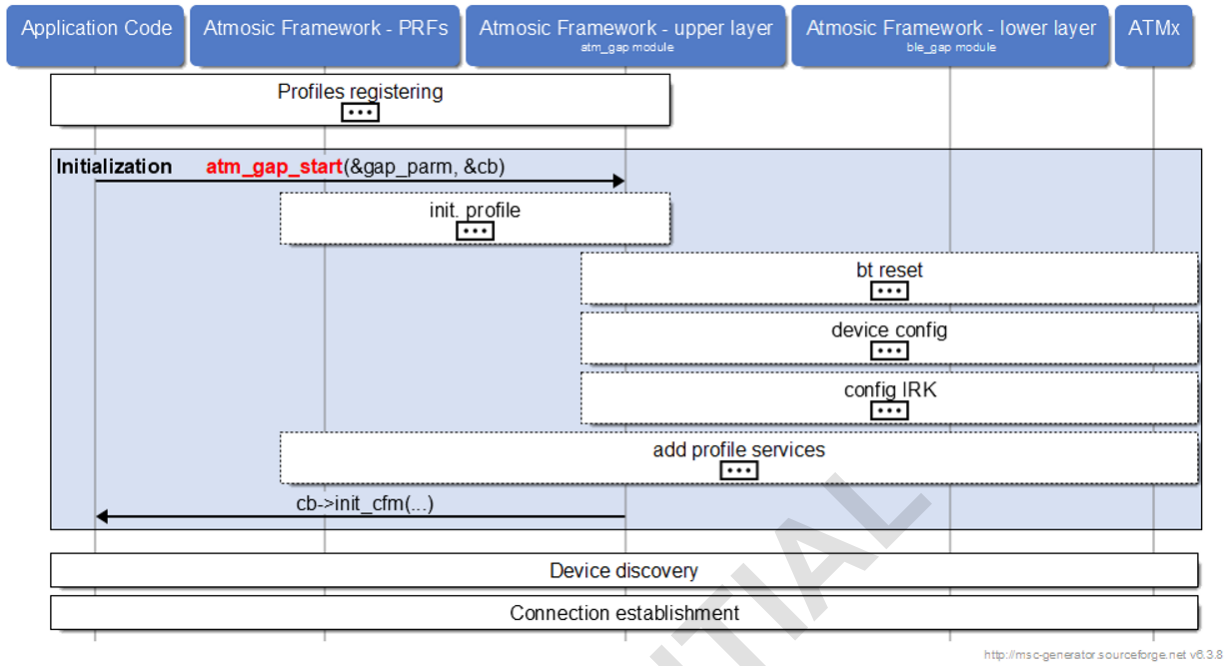



*Figure 4 - Initialization Process*

The following example shows how to configure the **atm_gap_param_t** and **atm_gap_cbs_t** in codes and call atm_gap_start for initialization in an application.

CONFIDENTIAL

```c
atm_gap_param_t gap_param= {
    .dev_name = dname,
    .dev_name_len = sizeof(dname),
    .dev_name_max = CFG_GAP_DNAME_MAX_LEN,
    .app_irk = CFG_GAP_FIX_IRK,
    .appearance = CFG_GAP_APPEARANCE,
    .slv_pref_params = {
        .con_intv_min = CFG_GAP_CONN_INT_MIN,
        .con_intv_max = CFG_GAP_CONN_INT_MIN,
        .slave_latency = CFG_GAP_SLAVE_LATENCY,
        .conn_timeout = CFG_GAP_CONN_TIMEOUT,
    },
    .dev_config = (struct gapm_set_dev_config_cmd*)&default_dev_conf,
    .addr = {
        .addr[0] = CFG_GAP_OWN_STATIC_RANDOM_ADDR0,
        .addr[1] = CFG_GAP_OWN_STATIC_RANDOM_ADDR1,
        .addr[2] = CFG_GAP_OWN_STATIC_RANDOM_ADDR2,
        .addr[3] = CFG_GAP_OWN_STATIC_RANDOM_ADDR3,
        .addr[4] = CFG_GAP_OWN_STATIC_RANDOM_ADDR4,
        .addr[5] = CFG_GAP_OWN_STATIC_RANDOM_ADDR5,
    },
};

const static atm_gap_cbs_t cb = {
    .conn_ind = _cb_conn_ind,
    .disc_ind = _cb_disc_ind,
    .pair_req_ind = _cb_pair_req_ind,
    .pair_passkey_ind = _cb_pair_passkey_ind,
    .pair_numeric_ind = _cb_pair_numeric_ind,
    .pair_ind = _cb_pair_ind,
    .conn_param_updated_ind = _cb_gap_conn_param_updated_ind,
    .init_cfm = _cb_gap_init_cfm,
    .quick_start_op = _cb_gap_quick_start_op,
};

...

static void _cb_gap_init_cfm(uint8_t status)
{
    if (status == GAP_ERR_NO_ERROR) {
            //Success
    } else {
    //Error
    }
}

static rep_vec_err_t _init(void)
{
    ...
    atm_gap_start(&gap_param, &cb);
    return (RV_DONE);
}
```

CONFIDENTIAL

## 2.4 Device Discovery - Advertising

In the device discovery process, the device could enter the advertising or scanning state to make it able to be found by the other devices or to scan other nearby devices.

The Atmosic application framework provides advertise modules to manage the most common Bluetooth LE advertising behavior and provide the flexible APIs to let developers adapt the Bluetooth LE advertising modes through these modules easily.

### 2.4.1 Atmosic Advertisement Modules

The atm_adv_param and atm_adv are the modules used by the Atmosic application framework. The atm_adv_param module exports the API to prepare the advertisement parameter for the atm_adv API. The advertising parameter can come from NVDS or predefined structure that can be overridden by using the define preprocessor. The parameters of advertisement are able to be changed and re-configured on the fly in application. See Figure 5.

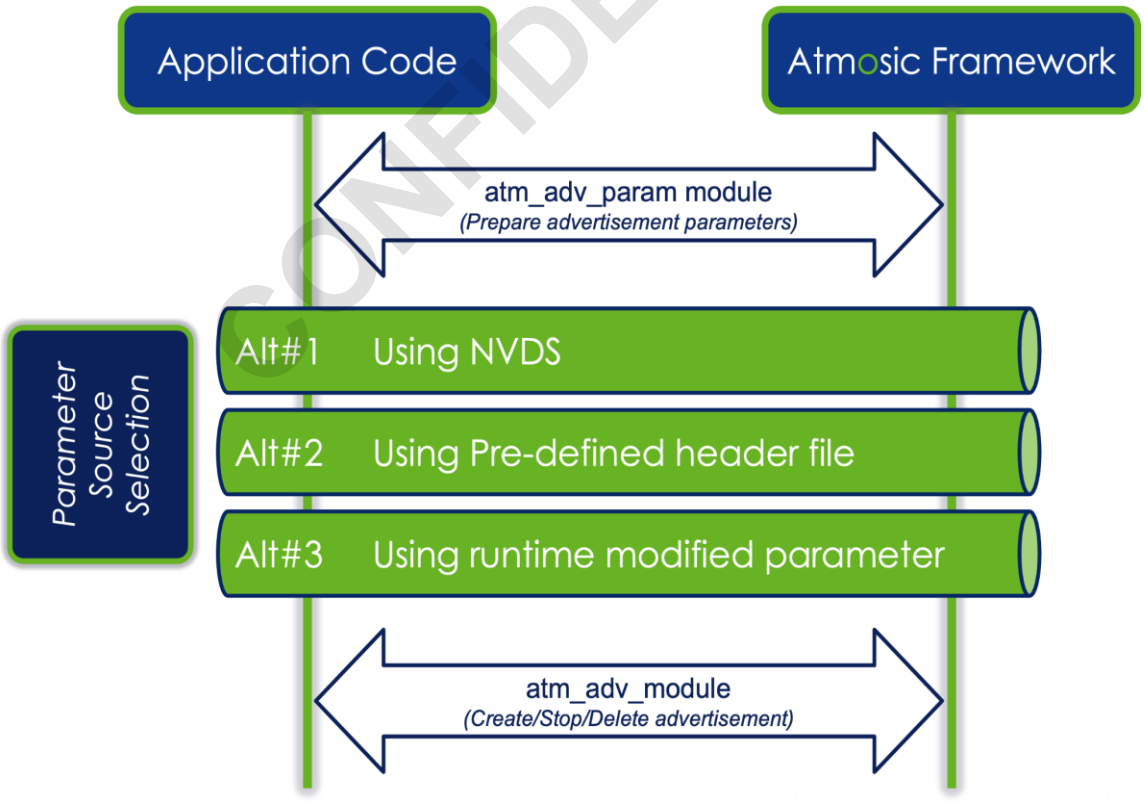



*Figure 5 - Advertisement Modules*

CONFIDENTIAL

### 2.4.2 atm_adv_param Module

There are four advertisement parameters to create advertisements. The atm_adv_param module exports the APIs to retrieve the parameters from NVDS or predefined structure.

Table 7 shows the parameters located in NVDS.

Table 7 - NVDS Parameters

| API Name | Parameter Purpose | Description | Parameters NVDS tag ID |
|---|---|---|---|
| atm_adv_create_param_nvds | Create parameter | Advertisement type<br>Discovery mode<br>Tx power<br>RF channel<br>configuration...etc | 0x06 |
| atm_adv_advdata_param_nvds | Adv. payload parameter | Advertisement payload | 0x0B |
| atm_adv_scandata_param_nvds | Scan response payload parameter | Scan response payload | 0x0C |
| atm_adv_start_param_nvds | Start parameter | Advertisement duration<br>Advertisement event counter | 0x05 |

CONFIDENTIAL

Table 8 shows the parameters located in predefined structure. Atmosic framework provides CFG_GAP_ADV_MAX_INST(1) instance number by default. Users can use makefile to overwrite the default instance number. The application code can use the instance number as input parameter when calling atm_adv_xxx_param_get API.

Table 8 - API Parameters Description

| API Name | Parameter Purpose | Description | Parameters Location |
|---|---|---|---|
| atm_adv_create_param_get | Create parameter | Configure:<br>advertisement type<br>discovery mode<br>tx power<br>channel<br>configuration...etc | default_adv_create_param of atm_adv_param.c |

| API Name | Parameter Purpose | Description | Parameters Location |
|---|---|---|---|
| atm_adv_advdata_param_get | Adv. payload parameter | Configure advertisement payload | default_set_adv_data of atm_adv_param.c |
| atm_adv_scandata_param_get | Scan response payload parameter | Configure scan response payload | default_set_adv_data of atm_adv_param.c |
| atm_adv_start_param_get | Start parameter | Configure:<br>advertisement duration<br>advertisement event counter | default_adv_start_param of atm_adv_param.c |

### Get parameter from NVDS

NVDS has defined four tag identities for the parameter used by advertisement. The application code can use the makefile and toolchain to build the NVDS data then burn into an Atmosic chip. The application code can use atm_adv_xxx_parameter_nvds to retrieve those settings before using atm_adv module to create the advertisement. See Figure 6.

CONFIDENTIAL

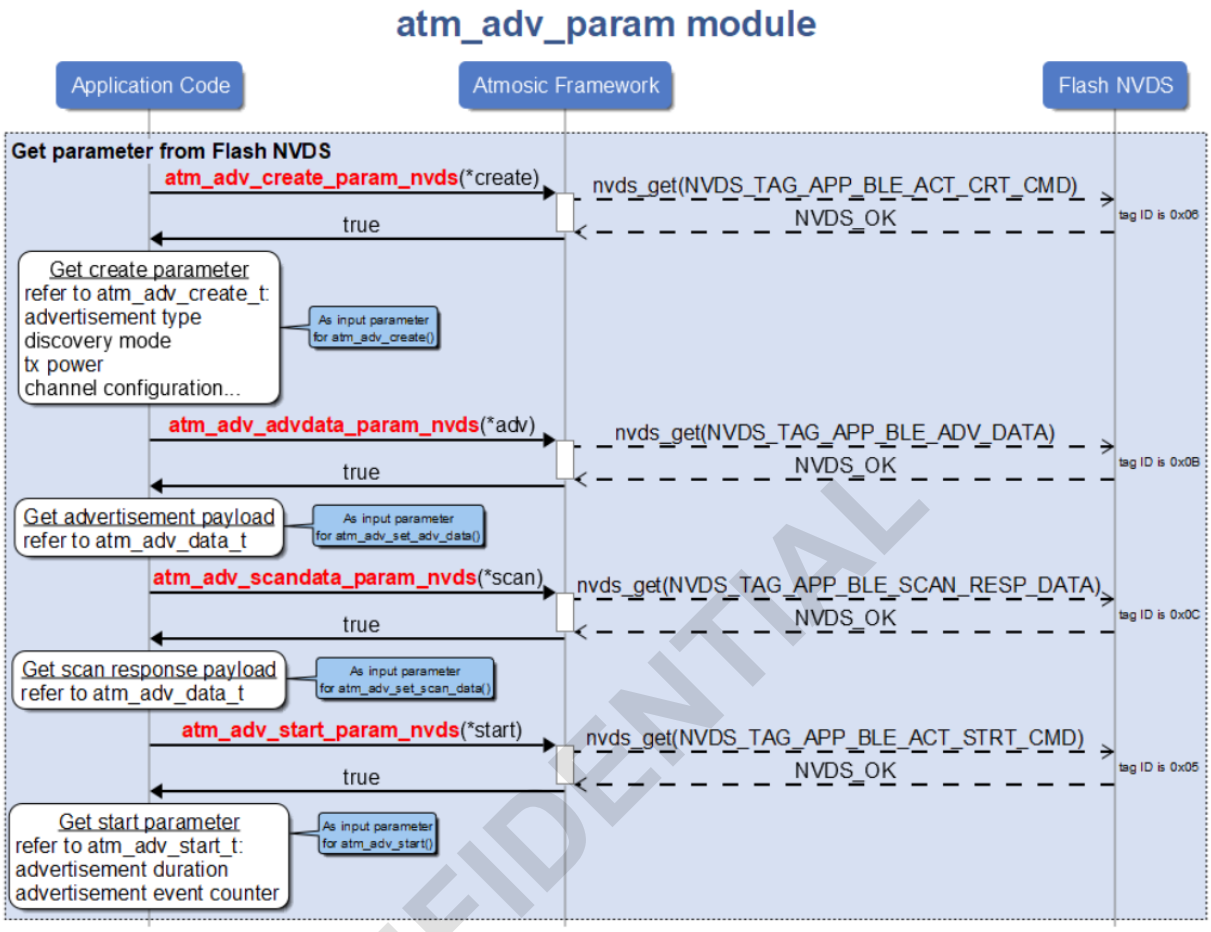



*Figure 6 - atm_adv_param Flow*

CONFIDENTIAL

## Get parameter from predefined structure

The predefined advertisement parameter instance is in the atm_adv_param module and also provides many overwrite fields to let the application code change the default value of predefined parameter instance. The parameters that can be overwritten are listed in atm_adv_param_internal.h. The application can apply the specific header file (-DGAP_PARM_NAME=”xxx.h” in makefile) to overwrite the parameter setting. See Figure 7.

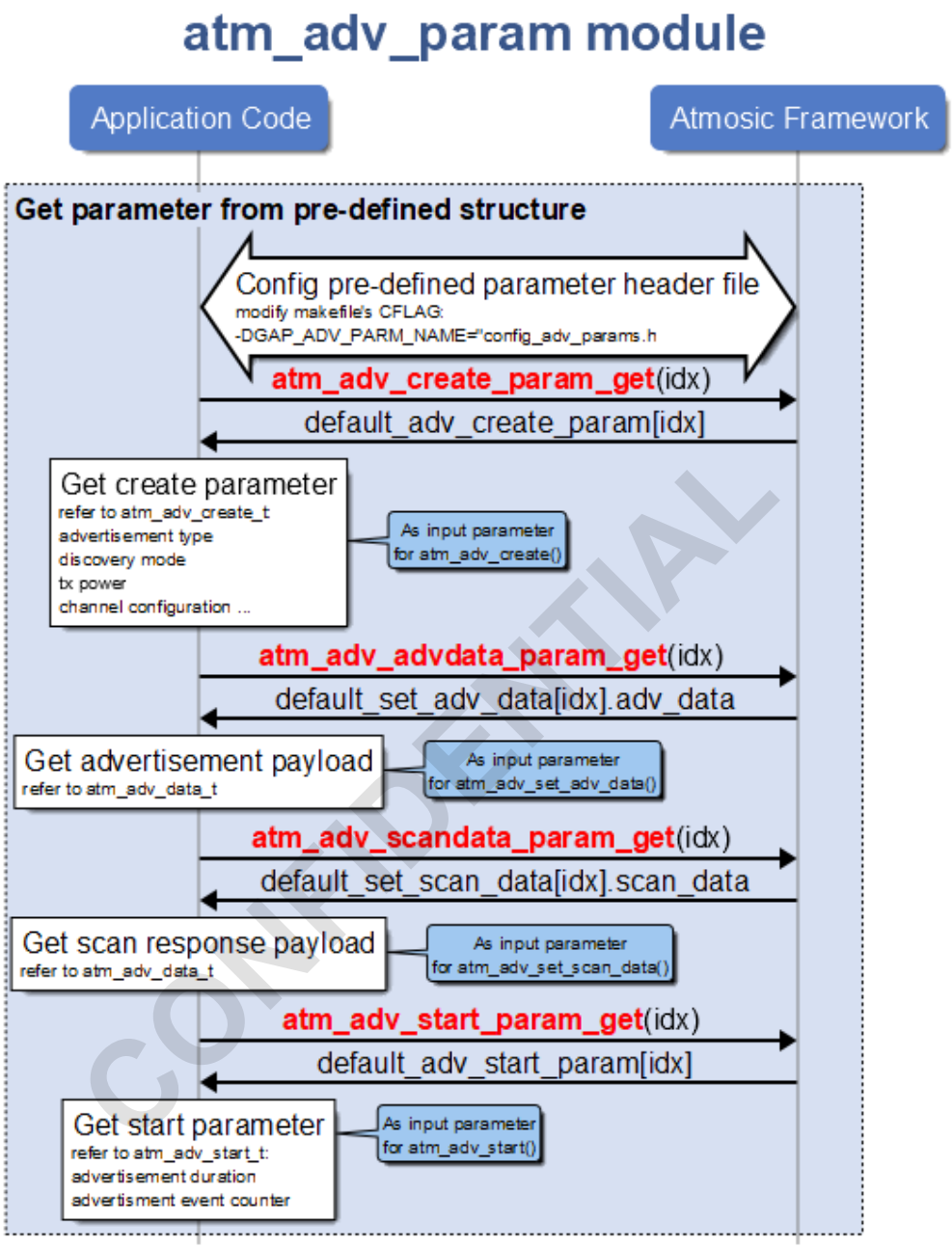



*Figure 7 - Get Parameter from Predefined Structure*

## Using runtime modified parameter

The default data type is constant in the predefined advertisement parameter instance. If the application code will need to change the parameter in some cases in runtime. The application code needs to remove the constant data type using "-DCFG_ADV_xxx_PARAM_CONST=0". See Figure 8.

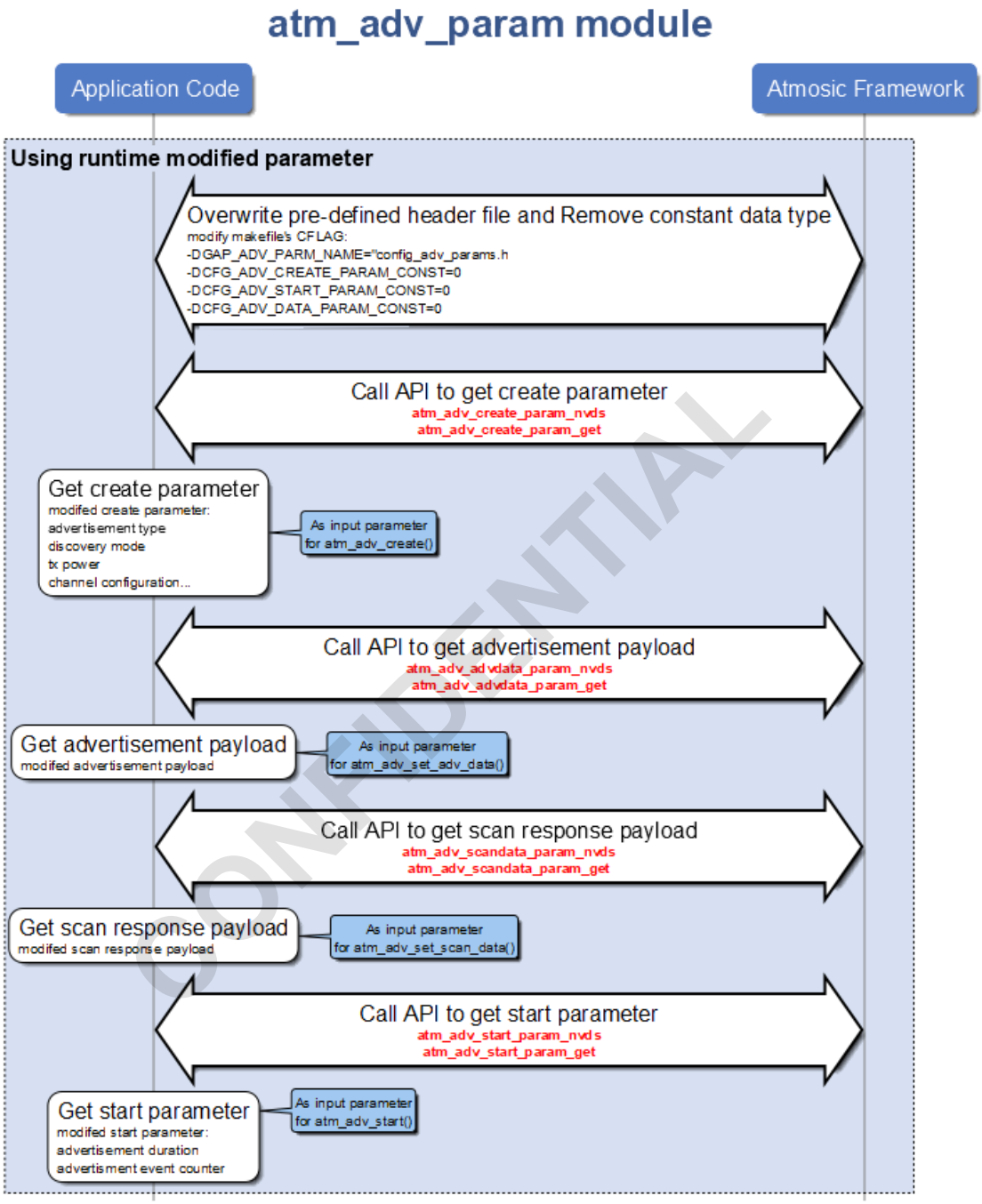



*Figure 8 - Using Runtime Modified Parameter*

### 2.4.3 atm_adv Module

After preparing the advertisement parameter datas via atm_adv_param module. The application will need to use the atm_adv module to control the advertisement. The application needs to provide the callback function that works with the atm_adv module. The atm_adv_reg is the first function call that provides the callback function. The atm_adv module will provide the event, activity index and status information for each

atm_adv exported API. The application code uses atm_adv_create API to create advertisement activity. The callback function will receive the ATM_ADV_CREATED event that includes the activity index. After getting the activity index value, the application code can use it as the input parameter for atm_adv exported API to control the advertisement. The application code can use atm_adv_create API to create multiple advertisement sets and uses the activity index to control specific advertisement sets.

The atm_adv_stop API will stop the advertisement transmission and the activity index is still valid for the atm_adv module. The application code will use atm_adv_start to re-enable the advertisement transmission again and doesn't need to call atm_adv_create and atm_adv_set_xxx_data again. The atm_adv module will destroy the activity instance when calling atm_adv_delete API. After this, the activity index will become invalid index for the atm_adv module.

Depending on advertising type, the advertisement and scan response payload will be used or not. The atm_adv_set_data_sanity API will perform sanity check before calling atm_adv_start API. See Figure 9 for the atm_adv flow.

CONFIDENTIAL

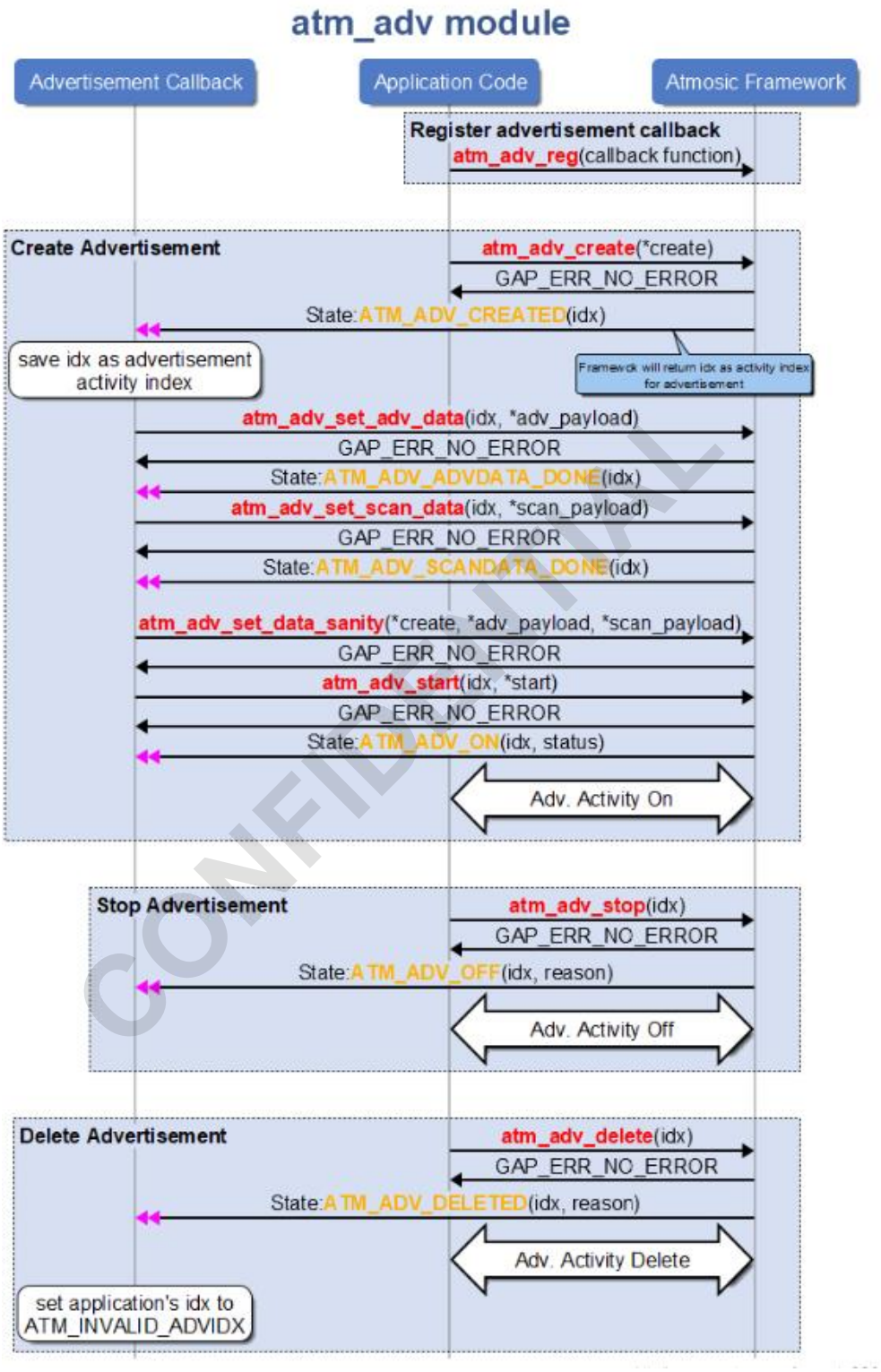



*Figure 9 - atm_adv Flow*

## 2.5 Device Discovery -Scanning

The Atmosic application framework provides scan modules to manage the Bluetooth LE scanning behaviors and provide the associated callback function to handle the scanning results.

### 2.5.1 Atmosic Scan Modules

The atm_scan_param and atm_scan are the modules used by the Atmosic application framework. The atm_scan_param module exports the API to prepare the scan parameter for the atm_scan API. The scanning parameter could be loaded from NVDS or a predefined instance that can be overridden by makefile with some particular preprocessor MACROs. These parameters can also be changed and re-configured by application during runtime. See Figure 10.

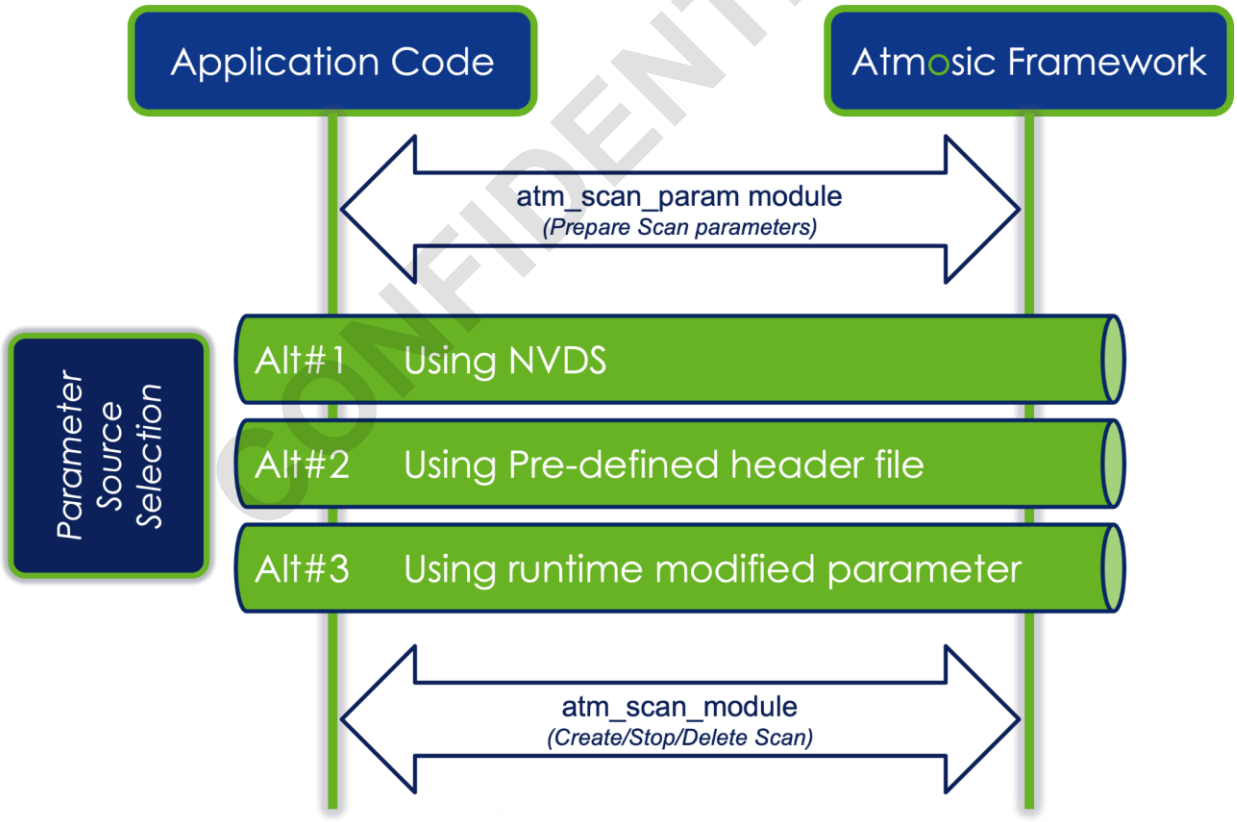



*Figure 10 - Scan Module Usage Diagram*

### 2.5.2 atm_scan_param Module

The overall behavior of the scanning activity is described by an aggregated parameter named atm_scan_params_t. This parameter includes both HW and SW settings such as scanning interval or the filtering policy for duplicated advertising packets. Since the

CONFIDENTIAL

SDK users would choose to use similar settings for most of their applications. The NVDS or predefined structure provides great reusability and flexibility for the SDK user to utilize.

### Get parameter from NVDS

NVDS has defined one tag identity for the parameter used by scan activity. The application code can use the makefile and toolchain to build the NVDS data then burn into an Atmosic chip. The application code can use atm_get_nvds_scan_params to retrieve the setting before using the atm_scan module to create the scan activity. See Figure 11.

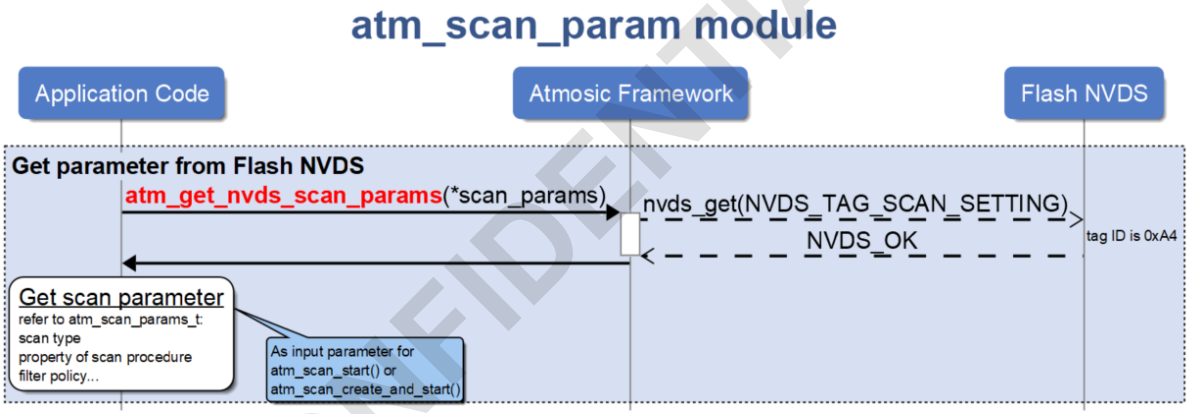



*Figure 11 - atm_scan_param Module Diagram*

CONFIDENTIAL

### Get parameter from predefined structure

The Atmosic framework provides CFG_GAP_SCAN_MAX_INST instances of a predefined scan parameter in the atm_scan_param module by default. Users can modify the number of instances in makefile and predefine the initial value of these instances for different scenarios as needed. The predefined scan parameter instance in the atm_scan_param module is initialized by several overwritable preprocessor macros. The SDK users may change these values by defining desired values in each application's makefile. The overwritable parameters are listed in atm_scan_param_internal.h. In addition to defining new values of those overwritable fields, the SDK users can also specify another header file in makefile (-DGAP_SCAN_PARM_NAME="xxx.h") to overwrite the entire setting for the parameter. See Figure 12.

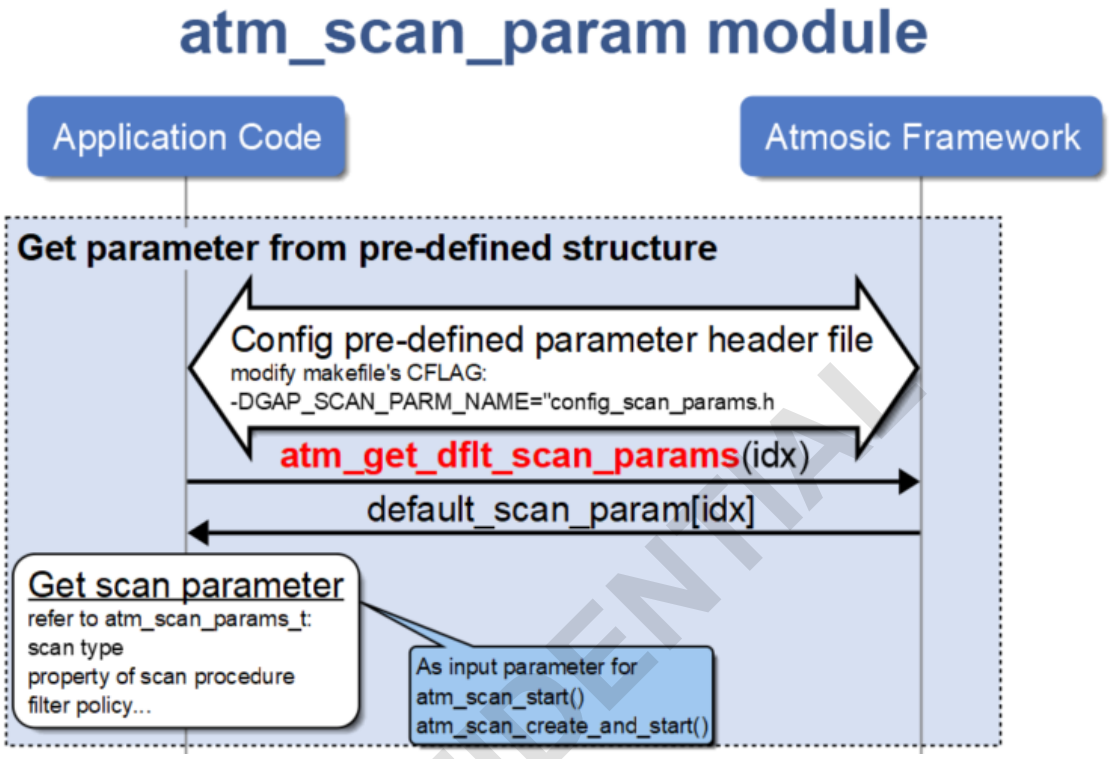



*Figure 12 - atm_scan_param Module Pre-defined Structure*

CONFIDENTIAL

### Using runtime modified parameter

In most scenarios the predefined parameters are loaded and directly sent into another API to start scanning activities without modifications. Thus the default data type is constant for the predefined scanning parameters. If the application code would need to change partial values of the parameter during runtime, the constant data type should be removed by adding “-DCFG_SCAN_PARAM_CONST=0” in the makefile of that application. See Figure 13.

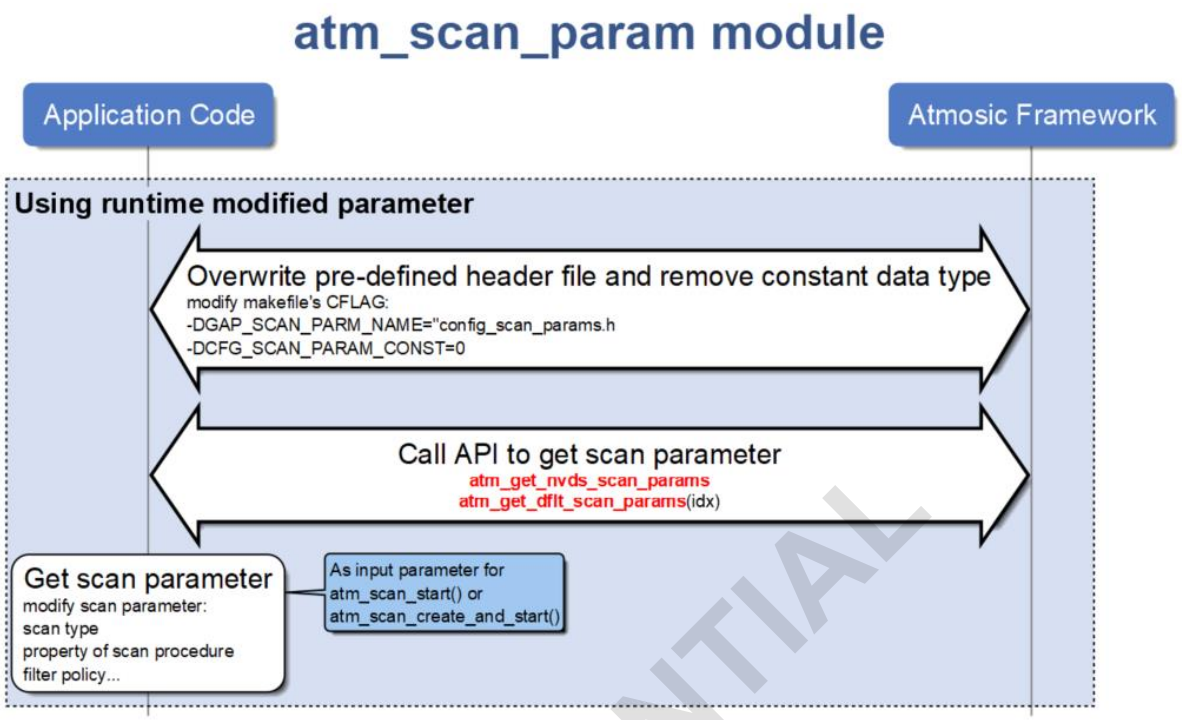



*Figure 13 - atm_scan_param Module Runtime Modified Parameter*

CONFIDENTIAL

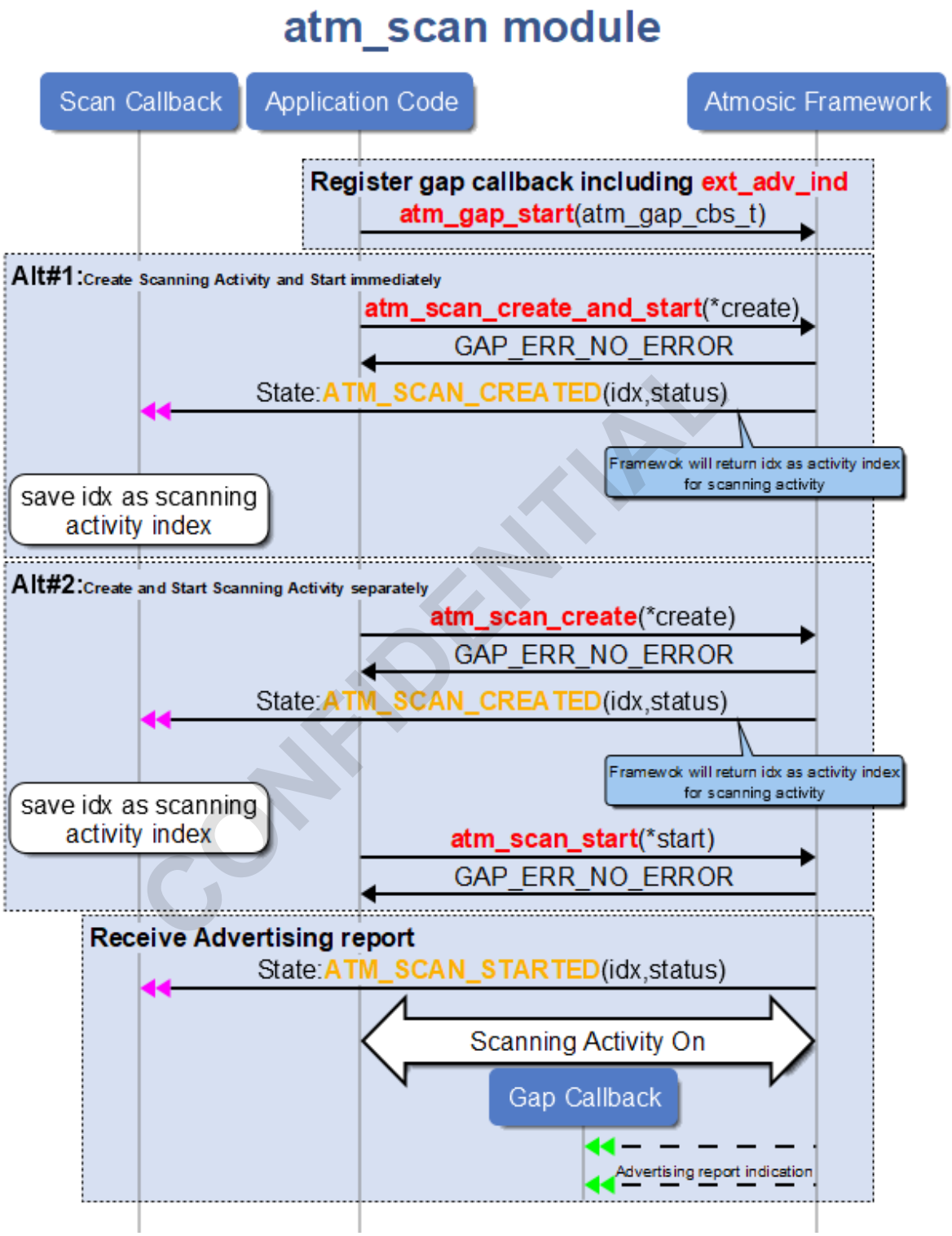
### 2.5.3 atm_scan Module

After preparing the scanning parameter with the methods introduced previously, the application can now utilize the atm_scan module to perform the scanning activity. The application needs to provide an extra callback function to handle the scanned advertising report in the function sets provided to the atm_gap module. Then the application code uses either atm_scan_create_and_start API to start scanning activity immediately or atm_scan_create to create scanning instance first then calls atm_scan_start to start scanning activity later. These two alternatives both expect the application code to provide a callback function set to handle the index of the scanning activity as well as the status change. Once the scanning activity is started successfully, the framework will start to send advertising reports to the gap callback function if there are BLE devices nearby.

If the duration of the scanning parameter is not zero, the scanning activity would stop automatically as specified. Otherwise the scanning activity would last permanently until the application code calls the atm_scan_stop API. The activity index is still valid for the atm_scan module even after the scanning activity is stopped. The application code could restart scanning by calling atm_scan_start again with the same activity index provided.

Once the scanning activity is not needed anymore, the application code should call atm_scan_delete API to destroy the scanning activity instance and release related resources in the Atmosic framework. The activity index will also become invalid thus the application code should clear the index cached locally. See Figure 14.

CONFIDENTIAL




CONFIDENTIAL

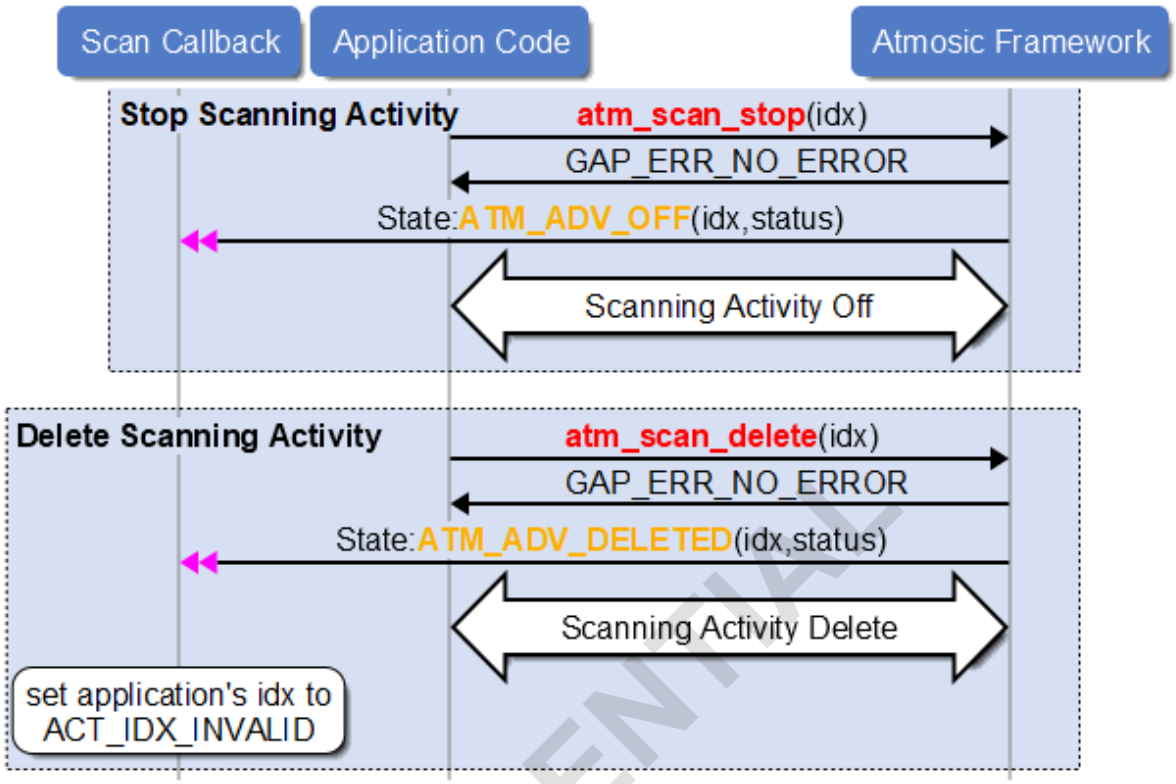



*Figure 14 - atm_scan Module Flow*

CONFIDENTIAL

## 2.6 Connection Establishment

The Atmosic application framework provides an initiator module to manage the Bluetooth LE initiating behaviors and provide the associated callback functions to handle the related events.For the controller, only one initiating procedure can be run. In the other word, it can not be allowed to establish two connections at the same time. In addition, it is needed to support the central role. Please check the role of gap with GAP_ROLE_CENTRAL.

### 2.6.1 Atmosic Initiator Modules

The atm_init_param and atm_init are the modules used by Atmosic application framework. The atm_init_param module exports the API to prepare the initiator parameter for the atm_init API. The initiator parameter can come from NVDS or predefined structure that can be overridden by using the define preprocessor. The parameters of the initiator are able to be changed and re-configured on the fly in application. See Figure 15.

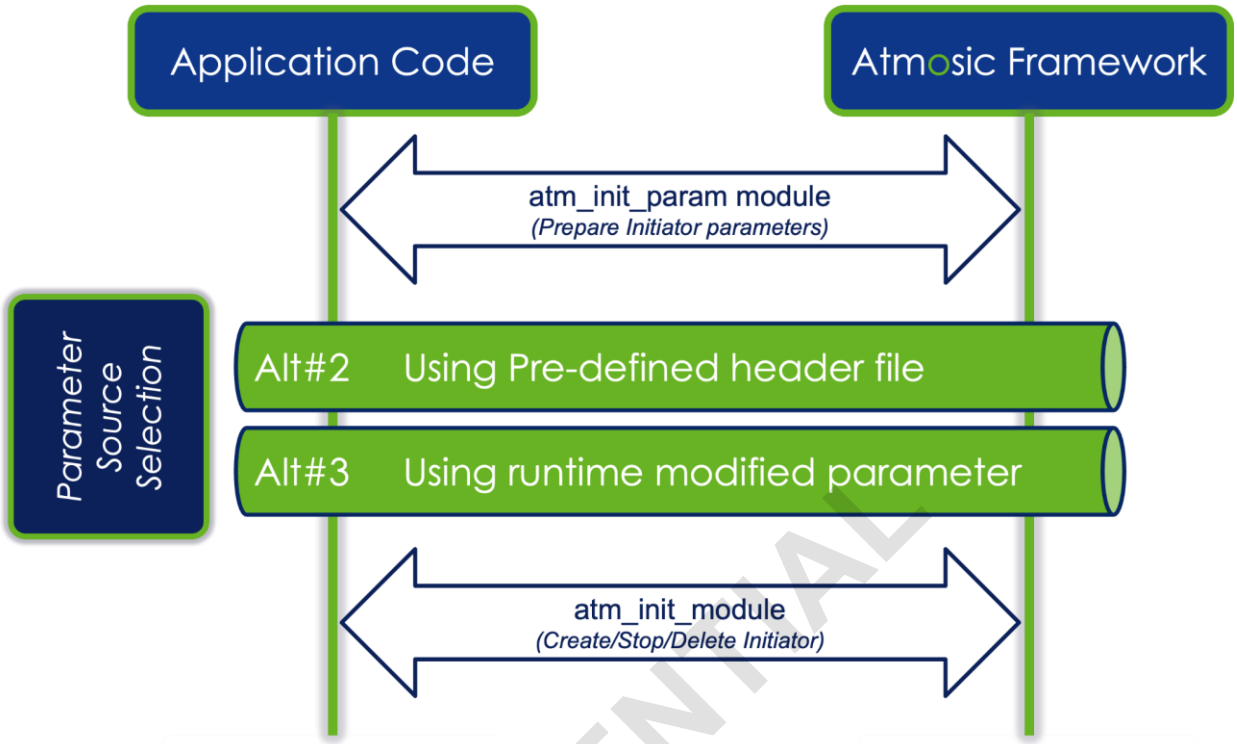



*Figure 15 - Initiator Modules*

### 2.6.2 atm_init_param Module

There is one kind of parameter to start the initiator. The atm_init_param module exports the APIs to retrieve the parameters from the predefined structure. See Table 9.

Table 9 - Initiator Parameters API Description

| API Name | Parameter Purpose | Description | Parameters Location |
|---|---|---|---|
| atm_init_param_get | Start parameter | Configure:<br>type<br>properties<br>connection timeout<br>connection interval<br>slave latency...etc | default_init_param of atm_init_param.c |

#### Get parameter from predefined structure

The predefined initiator parameter instance is in the atm_init_param module and also provides many overwrite fields to let the application code change the default value of the predefined parameter instance. The parameters that can be overwritten are listed

CONFIDENTIAL

into atm_init_param_internal.h. The application can apply the specific header file (-DGAP_INIT_PARM_NAME=”xxx.h” in makefile) to overwrite the parameter setting.

### Using runtime modified parameter

The default data type is not constant in the predefined initiator parameter  instance (CFG_INIT_PARAM_CONST is set to zero), so the application code can change the parameter in runtime. See Figure 16.

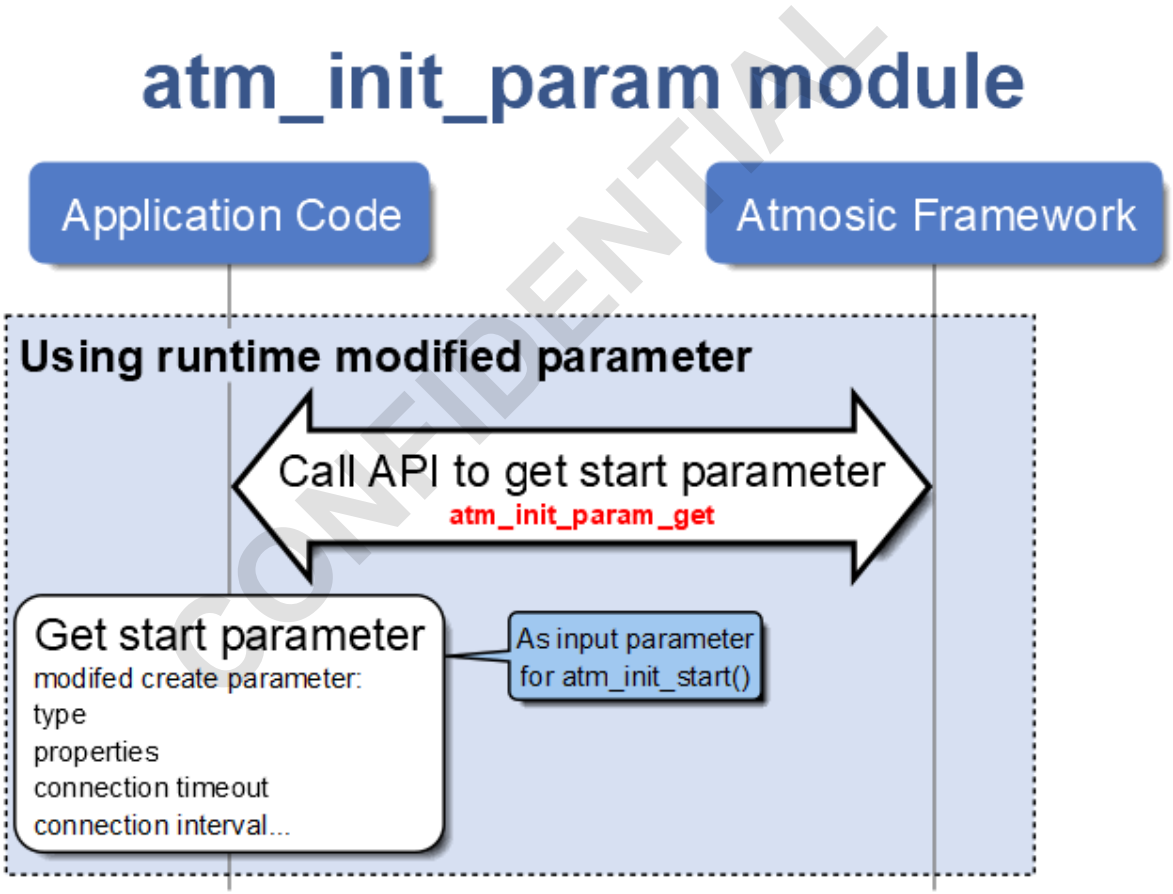



*Figure 16 - Initiator Parameter Module*

### 2.6.3 atm_init Module

After preparing the initiator parameter datas via atm_init_param module. The application will need to use atm_init module to control the initiator. For initiator, please set CFG_GAP_SCAN_MAX_INST value .The application needs to provide the callback function that works with atm_init module. The atm_init_reg is the first function call that

provides the callback function. The atm_init module will provide the state, activity index and status information for each atm_init exported API.

The application code uses atm_init_create API to create initiator activity. The callback function will receive the ATM_INIT_CREATED state that includes the activity index. After getting the activity index value, the application code can use it as the input parameter for atm_init exported API to control the initiator.

For establishing connections, the application layer can call the atm_init_start API and input the related parameter from atm_init_param modified the related parameters, ex: peer address for connecting device. The most important is that only one initiator can be executed. Don't execute two initiators at the same time. If receiving ATM_INIT_STARTING_FAIL state, the initiator would not start. If connected, the state would change to ATM_INIT_OFF. The activity index is still valid for atm_init module until invoking the atm_init_delete API.

The atm_init_stop API will stop the initiator and the activity index is still valid for atm_init module. The application code will use atm_init_start with the activity index to establish a new connection again and doesn't need to call atm_init_create again.

The atm_init module will destroy the activity instance when calling atm_init_delete API. After this, the activity index will become invalid index for the atm_init module.

CONFIDENTIAL

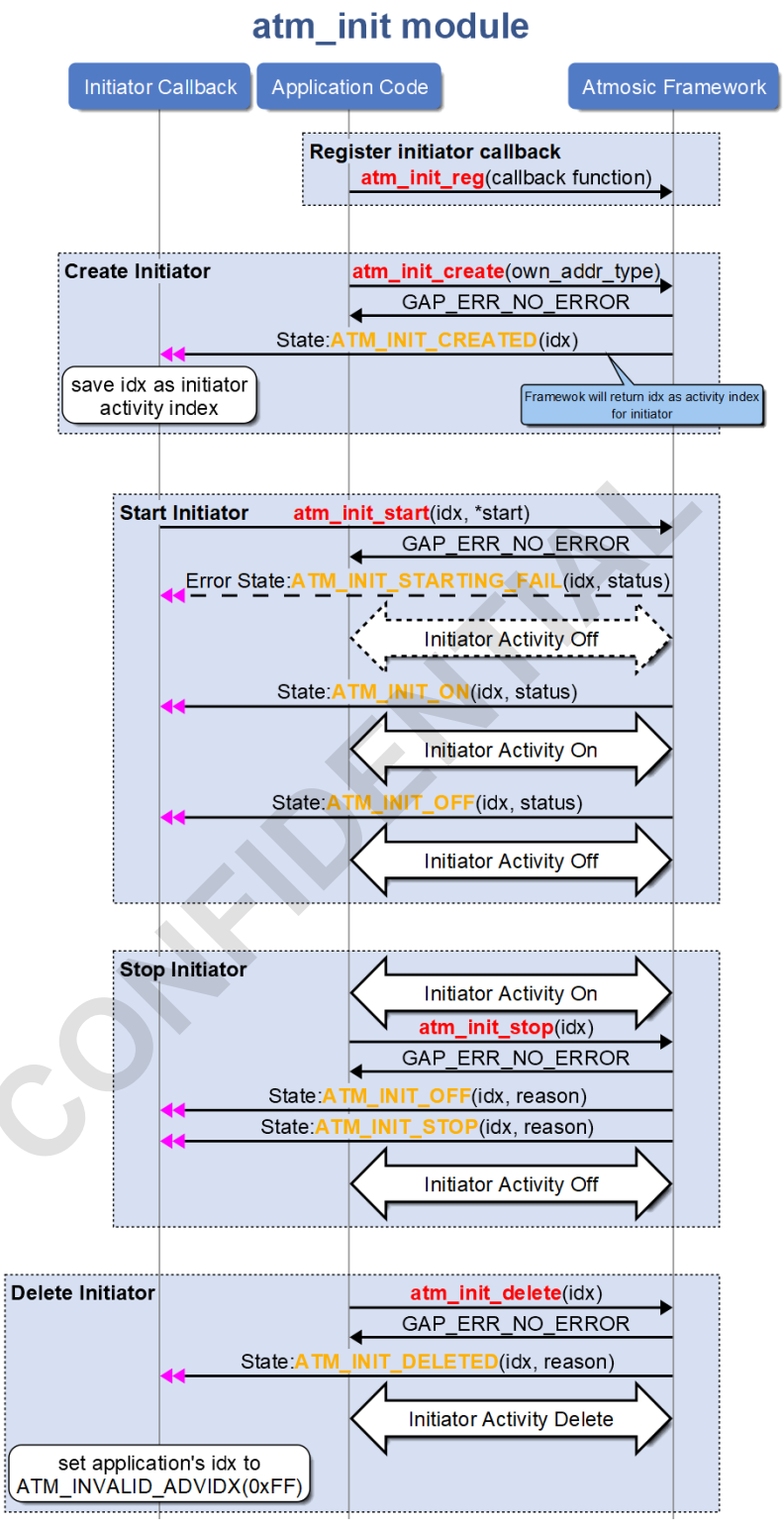



*Figure 17 - atm_init Flow*

CONFIDENTIAL

## 2.7 Connection Mechanism

In connection mechanism process, the applications could use atm_gap API functions and callbacks to handle the GAP messages from CEVA BLE stack. When a connection request indication event is received in atm_gap and the conn_ind callback is invoked in the application, the callback handler may call atm_gap_connect_accept to accept this connection request and keep handling the further indications with atm_gap API functions and callbacks.

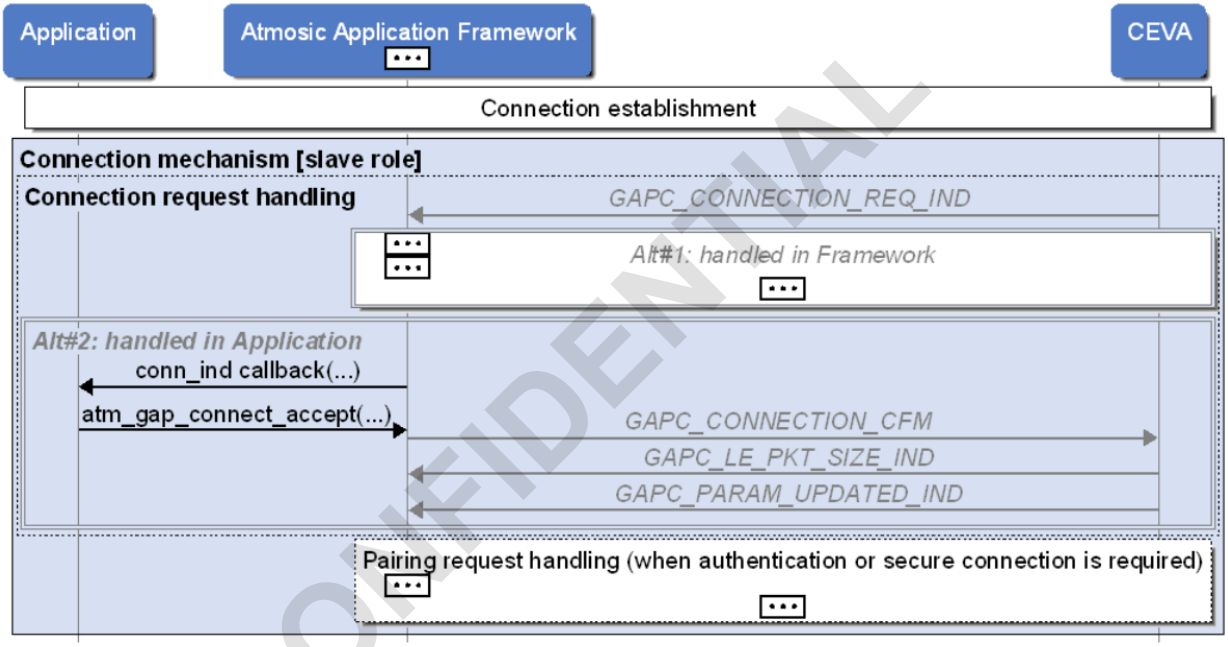



*Figure 18 - Connection Mechanism Sequence Overview*

CONFIDENTIAL

### 2.7.1 Detailed Bluetooth LE Connection Flow

Initialization sequence of the Bluetooth LE framework is shown in Figure 19. Atmosic Application Framework Connection Mechanism Sequence is shown in Figure 20.

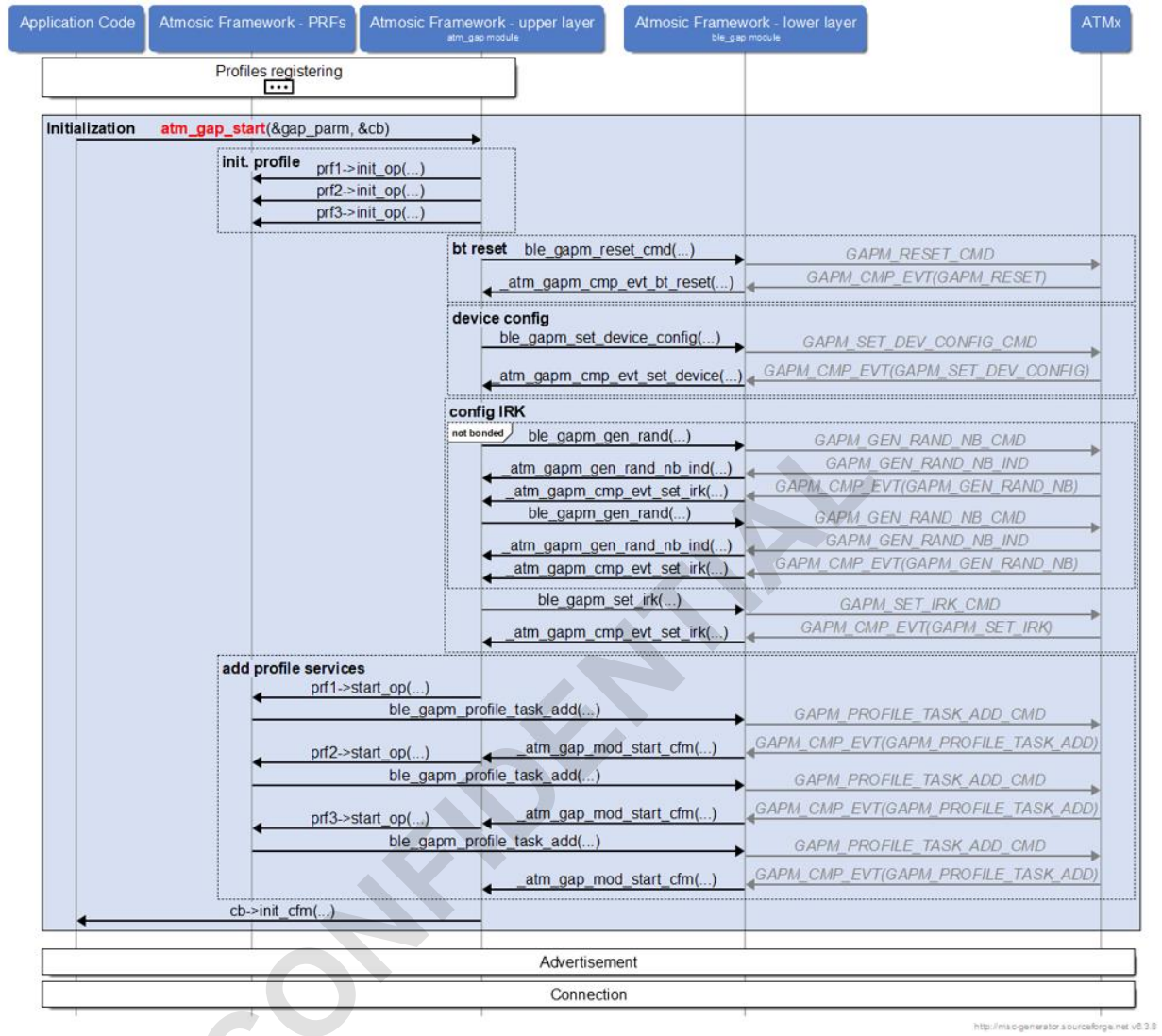



*Figure 19 - Atmosic Application Framework Initialization Sequence*

CONFIDENTIAL

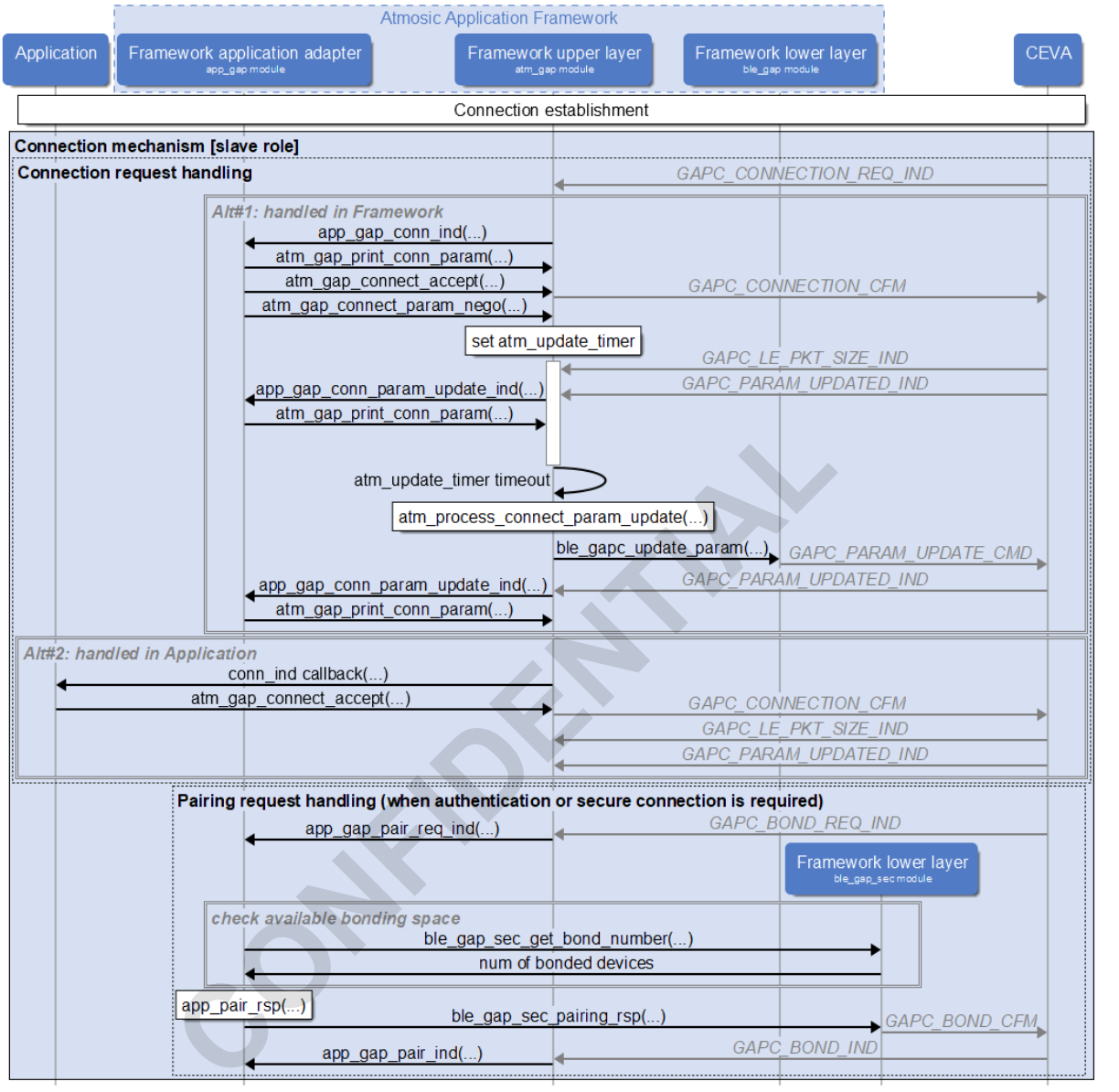



*Figure 20 - Atmosic Application Framework Connection Mechanism Sequence*

# 3 System Utilities

## 3.1 Application State Machine

Please see Application State Machine Application Note.

## 3.2 Log Utility

Application framework provides a log utility atm_log to manage and configure debug messages by each module (c source file).

### 3.2.1 ATM_LOG Macro

To use the atm_log utility, please include “atm_log.h” and call ATM_LOG_LOCAL_SETTING macro to configure the debug level and module name in each source file and then use ATM_LOG macro to print the different level debug messages.

- ATM_LOG_LOCAL_SETTING(name, level)
- ATM_LOG(level, dbg_msg, ...)

ATM_LOG debug message format:

@{timestamp} [{name}][{level}]: {dbg_msg}

CONFIDENTIAL

### 3.2.2 Debug Log Level

Table 10 lists all debug log level masks which are used in the ATM_LOG macros.

Users could use -DATMLOG_GLOBAL_LEVEL={debug_level_mask} to configure the debug level of the whole application.

Table 10 - Debug Log Level

| Debug Level Mask | Log Type | Description |
|---|---|---|
| V | Verbose log | When this debug level mask is set in the local setting, all log types will be output in the module. |
| D | Debug log | When this debug level mask is set in the local setting, the Debug, Notify, Warning and Error log types will be output in the module. |
| N | Notify log | When this debug level mask is set in the local setting, the Notify, Warning and Error log types will be output in the module. |

| Debug Level Mask | Log Type | Description |
|---|---|---|
| W | Warning log | When this debug level mask is set in the local setting, the Warning and Error log types will be output in the module. |
| E | Error log | When this debug level mask is set in the local setting, only the Error log type will be output in the module. |

***Note**: The length of the name display is up to 10. The oversized characters will be truncated.*

### 3.2.3 How to Use

The following is an ATM_LOG macros use example:

C source file

```c
#include "atm_log.h"

ATM_LOG_LOCAL_SETTING("modulename", D);

static void module_test(void)
{
    ATM_LOG(D, "enter %s", __func__);
    ATM_LOG(W, "exit %s", __func__);
}
```

The debug messages would be printed as below in the serial terminal software which supports ANSI escape code.

```
@0016f0e0 [modulename][D]: enter module_test
@0016f186 [modulename][W]: exit module_test
```

For the users who just need a simple debug function with timestamp only and ignore the debug level setting, please use the marco below:

DEBUG_TRACE(dbg_msg, ...)

CONFIDENTIAL

Debug trace message format:

@{timestamp} {dbg_msg}

## 3.3 Power Management

Application framework provides a power manager module atm_pm to help users to manage the power mode of the device.

### 3.3.1 Power Saving Modes

ATM2/ATM3 devices support 3 types of power saving mode as shown in Table 11.

Table 11 - Power Saving Modes

| Power Saving Mode | Description |
|---|---|
| Sleep Mode | All variables and registers would be kept. The power consumption is the highest in 3 power saving modes. |
| Retention Mode | Drop to retention voltage, retain the required system blocks only. The power consumption is in the middle of 3 power saving modes. |
| Hibernation Mode | Only specific registers would be kept. The power consumption is the lowest and close to SoC OFF mode. It looks like a reboot when the system wakes up from hibernation mode. |

CONFIDENTIAL

### 3.3.2 Power Mode Lock

ATM2/3 devices will always try to enter the lowest power consumption mode when the system is idle. However, there are some activities that might still be on-going or wait for the response. To prevent the system unexpectedly going to power saving mode during the activities, atm_pm uses the locks for different power saving modes to control the system behavior. Figure 21 illustrates the power mode control with the atm_pm locks.

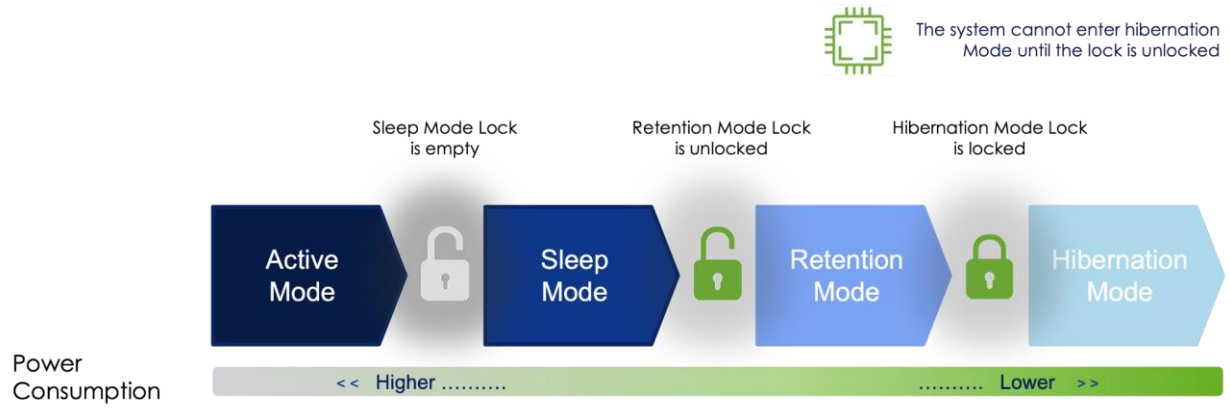



*Figure 21 - Power Mode Lock*

CONFIDENTIAL

### 3.3.3 Power Manager API

Table 12 lists all power manager APIs.

Table 12 - Power Manager API

| Power Manager API | Description |
|---|---|
| pm_lock_id_t atm_pm_alloc(pm_lock_type_e type) | Allocate lock identifier with lock type |
| bool atm_pm_realloc(pm_lock_id_t id, pm_lock_type_e type) | Allocate lock identifier with specified id and type |
| void atm_pm_free(pm_lock_id_t id) | Free allocated lock |
| void atm_pm_lock(pm_lock_id_t index) | Lock specific power mode |
| void atm_pm_unlock(pm_lock_id_t index) | Unlock specific power mode |
| void atm_pm_lock_info(void) | Print all locks status |
| void atm_pm_set_hib_restart_time(uint32_t restart_time) | Set restart time from hibernate |
| [ATM33/e]<br>void atm_pm_set_hibernate_cb (rep_vec_fn__ret_bool__int32_t__uint32_t_p__t cb)<br><br>[ATM2/ ATM3]<br>void atm_pm_set_hibernate_cb (rep_vec_fn__ret_bool__int32_t__uint32_t__t cb) | Set function of atm_pm replacement vector of hibernate |

### 3.3.4 atm_pm lock example

The following is a simple atm_pm lock example:

makefile

```
DRIVERS := interrupt timer atm_pm sw_timer sw_event
```

C source file

```c
#include "atm_pm.h"

static pm_lock_id_t lock_hiber;

static void user_init(void)
{
    ...
    lock_hiber = atm_pm_alloc(PM_LOCK_HIBERNATE);
    atm_pm_lock(lock_hiber);
    ATM_LOG(D, "Hibernation is locked.");
    // Prevent the system entering hibernation.
    ...
}

static void activity_stop(void)
{
    atm_pm_unlock(lock_hiber);
    ATM_LOG(D, "Hibernation is unlocked.");
    // System goes into hibernation.
}
```

CONFIDENTIAL

Please refer to pm_demo example in the SDK for more atm_pm use examples.

## 3.4 Software Timer

Application framework provides software timer functionality through sw_timer module. Users can use sw_timer API to alloc, set and clear timers for their application.

### 3.4.1 Software Timer API

Table 13 lists all sw_timer APIs.

Table 13 - Software Timer APIs

| Software Timer API | Description |
|---|---|
| sw_timer_id_t sw_timer_alloc (sw_timer_func_t handler, const void *ctx) | Allocate and configure timer |
| void sw_timer_free(sw_timer_id_t timer_id) | Free allocated timer |
| void sw_timer_reconfig (sw_timer_id_t timer_id, sw_timer_func_t handler, const void *ctx) | Reconfigure timer handler and context |
| void sw_timer_set(sw_timer_id_t timer_id, uint32_t centisec) | Start one-shot timer running |
| void sw_timer_clear(sw_timer_id_t timer_id) | Stop/abort running timer |
| bool sw_timer_active(sw_timer_id_t timer_id) | Get timer status |

The time unit of sw_timer_set is cent-second (10 ms). A "SW_TIMER_ID_MAX too big for sw_timer_id_t" assertion might occur when the users try to allocate more than 8 software timers. By default, the number of software timers is limited to 8 by SW_TIMER_ID_MAX. Users could change the value in makefile with -DSW_TIMER_ID_MAX=N to enlarge the maximum number of software timers based on their application needs.

### 3.4.2 sw_timer example

The following is a simple sw_timer example:

makefile

```
1 DRIVERS := interrupt timer atm_pm sw_timer sw_event
```

C source file

CONFIDENTIAL

```c
#include "sw_timer.h"

static sw_timer_id_t tid_test;
static bool repeat;

static void timeout_handler(sw_timer_id_t tid, void const *ctx)
{
    ATM_LOG(D, "Timer expired.");
    if (repeat) {
        sw_timer_set(tid_test, 5 * SW_TIMER_1_SEC);
    }
}

static void user_init(void)
{
    ...
    tid_test = sw_timer_alloc(timeout_handler, NULL);
    sw_timer_set(tid_test, 5 * SW_TIMER_1_SEC);
    ATM_LOG(D, "Timer started.");
    ...
}
```

CONFIDENTIAL

## 3.5 Software Event

Application framework provides software event functionality through sw_event module. Users could use sw_event APIs to allocate, set and clear software events for their application.

### 3.5.1 Software Event API

Table 14 lists all sw_event APIs.

Table 14 - Software Event APIs

| Software Event API | Description |
|---|---|
| sw_event_id_t sw_event_alloc (sw_event_func_t handler, const void *ctx) | Allocate and configure event |
| void sw_event_free(sw_event_id_t event_id) | Free allocated event |
| void sw_event_reconfig (sw_event_id_t event_id, sw_event_func_t handler, const void *ctx) | Reconfigure event handler and context |

| Software Event API | Description |
| --- | --- |
| void sw_event_set(sw_event_id_t event_id) | Trigger event |
| void sw_event_clear(sw_event_id_t event_id) | Clear event |
| bool sw_event_get(sw_event_id_t event_id) | Get event status |

A "SW_EVENT_ID_MAX too big for sw_event_mask" assertion might occur when the users try to allocate more than 8 software events. By default, the number of software events is limited to 8 by SW_EVENT_ID_MAX. Users could change the value in makefile with -DSW_EVENT_ID_MAX=N to enlarge the maximum number of software events based on their application needs.

### 3.5.2 sw_event example

The following is a simple sw_event example:

makefile

```
1 DRIVERS := interrupt timer atm_pm sw_timer sw_event
```

C source file

CONFIDENTIAL

```c
#include "sw_event.h"

static sw_event_id_t eid_test;

static void event_handler(sw_event_id_t eid, void const *ctx)
{
    bool done = false;
    ATM_LOG(D, "Handling the event ... ");
    ...
    if (done) {
        sw_event_clear(eid);
        ATM_LOG(D, "Done. Clear the event.");
    }
    ...
}

static void user_init(void)
{
    ...
    eid_test = sw_event_alloc(event_handler, NULL);
    ...
}

static void trigger_event(void)
{
    sw_event_set(eid_test);
    ATM_LOG(D, "Event triggered.");
}
```

CONFIDENTIAL

## 3.6 AT Command

Please see the Atmosic AT Command User Guide (available on the Atmosic support website).

# 4. Hardware Drivers

## 4.1 GPIO Driver

The Atmosic application framework provides atm_gpio modules to provide the associated API function to control the pins for GPIOs application.

### 4.1.1 atm_gpio Module

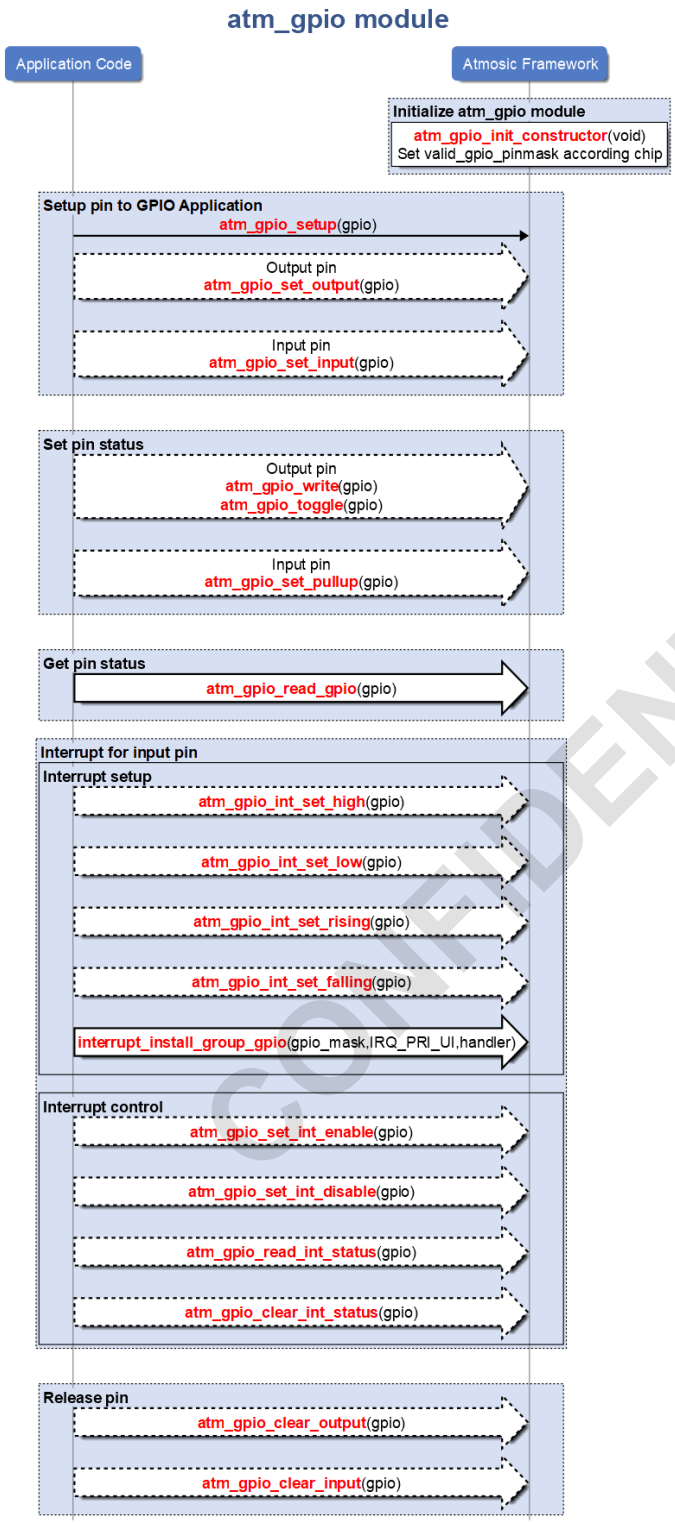
To enable the atm_gpio module, please add the module to DRIVERS in makefile.

```
DRIVERS := \
    atm_ble \
    atm_button  \
    atm_gpio \
    atm_pm \
```

As initializing the atm_gpio module, the atm_gpio_init_constructor would be executed and valid io would be marked according to the chip type to avoid setting invalid io.

For GPIO applications, the applications could use atm_gpio API functions to control or get the output and input status. For input pin application, it also can support the interrupt handling.  The related setting flow and APIs are listed as below:

CONFIDENTIAL




CONFIDENTIAL

*Figure 22 - atm_gpio Module Flow*

The atm_gpio module exports the APIs to set pins as GPIOs and control/retrieve the status of GPIOs. See Table 15.

Table 15 - Atmosic GPIO API Description

| API Name | Purpose | Parameter Description |
|---|---|---|
| void atm_gpio_setup(uint8_t gpio) | Set pin as GPIO | gpio: The number of GPIO not pin.<br>EX: For P20, the related GPIO is 16.<br>atm_gpio_setup(16);<br><br>The related mapping please refer to section 2.7 Pin Multiplexing in ATM3_ATM2 Reference Manual. |
| void atm_gpio_set_input(uint8_t gpio)/<br>void atm_gpio_clear_input(uint8_t gpio) | Set/Clear gpio as input I/O | gpio: The number of GPIO |
| void atm_gpio_set_output(uint8_t gpio)/<br>void atm_gpio_clear_output(uint8_t gpio) | Set/Clear gpio as output I/O | gpio: The number of GPIO |
| void atm_gpio_set_pullup(uint8_t gpio)/<br>void atm_gpio_clear_pullup(uint8_t gpio) | Set/Clear gpio with pullup | gpio: The number of GPIO |
| void atm_gpio_write(uint8_t gpio, bool value) | Set output high or low for output I/O | gpio: The number of GPIO<br>value: 0-low 1:high |
| bool atm_gpio_read_gpio(uint8_t gpio) | Read the status for GPIO | gpio: The number of GPIO<br>return: the status of the GPIO |
| bool atm_gpio_toggle(uint8_t gpio) | Toggle the output | gpio: The number of GPIO<br>return: the status of the GPIO after toggle |

CONFIDENTIAL

## 4.2 I2C Driver

There are two identical I2C master ports, and it supports clock frequencies between 3.9 kHz~4 MHz. The Atmosic application framework provides i2c modules with the associated APIs for i2c communication.

### 4.2.1 i2c Module

To enable the atm_gpio module, please add the module to DRIVERS in makefile.

```
DRIVERS := \
    atm_ble \
    atm_button  \
    i2c \
```

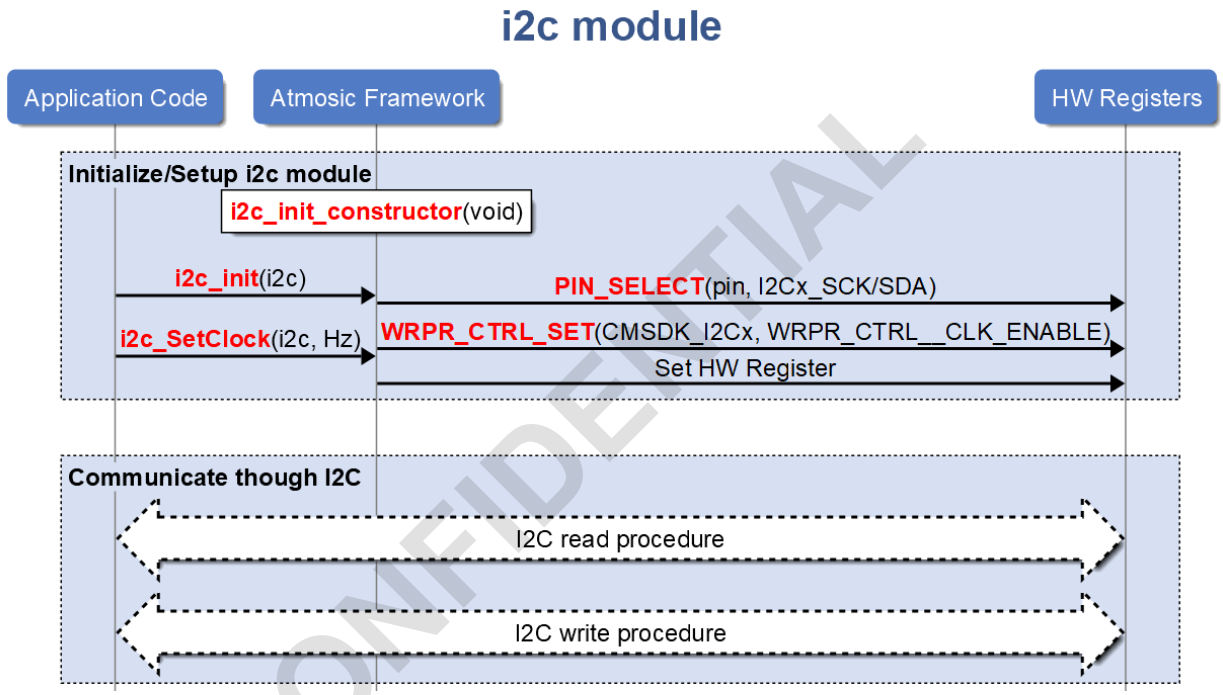



*Figure 23 - i2c Module Flow*

CONFIDENTIAL

When initializing the i2c module, the i2c_init_constructor would be executed to register related callback function to control the related pin as enter/exit retention or hibernation mode.

The input i2c_dev_t for the I2C APIs can be configured according to the I2C set and default status of the data pin. The CMSDK_AT_APB_I2C_TypeDef can be selected to CMSDK_I2C0 or CMSDK_I2C1. enable_data_pullup is used to set the default status of the data pin.

```c
typedef struct i2c_dev_s {
    CMSDK_AT_APB_I2C_TypeDef *base;
    bool enable_data_pullup;

    int rx_head, rx_tail;
    uint8_t rx_buf[I2C_BUF_SIZE];

    int tx_head, tx_tail;
    uint8_t tx_address;
    uint8_t tx_buf[I2C_BUF_SIZE];
} i2c_dev_t;
```

i2c_init is the initialization function for I2C. It would set the I2C pin assignment according to the chip and below define. The default setting is defined in i2c.c, and it can be changed by defining the setting in makefile. For the pin multiplexing, please refer to section 2.7 of the Pin multiplexing in ATM3_ATM2 Reference Manual (available on the Atmosic support website).

CONFIDENTIAL

```
#ifndef CFG_5×5_I2C0_SCK
#define CFG_5×5_I2C0_SCK 9
#endif

#ifndef CFG_5×5_I2C0_SDA
#define CFG_5×5_I2C0_SDA 10
#endif

#ifndef CFG_5×5_I2C1_SCK
#define CFG_5×5_I2C1_SCK 11
#endif

#ifndef CFG_5×5_I2C1_SDA
#define CFG_5×5_I2C1_SDA 13
#endif

#ifndef CFG_6×6_I2C0_SCK
#define CFG_6×6_I2C0_SCK 29
#endif

#ifndef CFG_6×6_I2C0_SDA
#define CFG_6×6_I2C0_SDA 30
#endif

#ifndef CFG_6×6_I2C1_SCK
#define CFG_6×6_I2C1_SCK 12
#endif

#ifndef CFG_6×6_I2C1_SDA
#define CFG_6×6_I2C1_SDA 13
#endif
```

CONFIDENTIAL

For I2C clock frequency, it can be set by the i2c_SetClock.

```
EX: I2C 0, pullup data pin, clock frequency: 100KHz
static i2c_dev_t i2c_dev = {
    .base = CMSDK_I2C0,
    .enable_data_pullup = true
};
#define I2C_CLK_100K 100000

i2c_init(&i2c_dev);
i2c_SetClock(&i2c_dev, I2C_CLK_100K);
```

For the i2c module, there are APIs for I2C communication steps and it can be composed to a procedure for a master to access a slave device.

Table 16 - Atmosic I2C API Description

| API Name | Purpose | Parameter Description |
|---|---|---|
| void i2c_init(i2c_dev_t *i2c) | Initialize I2C device structure | i2c: device structure |
| void i2c_SetClock(i2c_dev_t *i2c, uint32_t Hertz) | Configure I2C clock frequency | i2c: device structure<br>Hertz: clock frequency |
| int i2c_requestFrom(i2c_dev_t *i2c, uint8_t address, int quantity, bool stop) | Initiate read transaction from I2C slave device | i2c: device structure<br>address: i2c slave address<br>quantity: Number of bytes to read from slave.<br>stop: Signal stop after transaction<br>return: Success: number of bytes requested from bus. Failure: negative i2c_et_t failure code |
| int i2c_available(i2c_dev_t *i2c) | Poll status of current read transaction | i2c: device structure<br>Number of bytes available to read via i2c_read() |
| uint8_t i2c_read(i2c_dev_t *i2c) | Read next byte of data. | i2c: device structure<br>return: Next byte of data |
| void i2c_beginTransmission(i2c_dev_t *i2c, uint8_t address) | Initiate write transaction to I2C slave. | i2c: device structure<br>address: i2c slave address |
| int i2c_write_byte(i2c_dev_t *i2c, uint8_t value) | Append single byte to current write transaction | i2c: device structure<br>value: write data<br>return:Success: 1 Failure: 0 |
| int i2c_write_block(i2c_dev_t *i2c, const uint8_t *data, int length) | Append bytes to current write transaction | i2c: device structure<br>data: Block of bytes to append.<br>int: length<br>return: Number of bytes actually added. |
| i2c_et_t i2c_endTransmission(i2c_dev_t *i2c, bool stop) | Finalize current write transaction and wait for completion. | i2c: device structure<br>stop: Signal stop after transaction<br>return: error codes |

In the SDK, there are some sensor drivers using I2C, ex: adt7420/bme680/lis2dh12. Those drivers can be reference code for I2C applications. In the section, use lis3dh as an example for I2C read / write procedure .

CONFIDENTIAL

The lis3dh_read API is to read data from the lis3dh register. First, write one byte data for the register address and then readback the data.

```
1 static int lis2dh12_read(uint8_t reg_addr, uint8_t *data, int length)
2 {
3     i2c_beginTransmission(&i2c_dev,I2C_HDC2010_ADDR);
4     i2c_write_byte(&i2c_dev, reg_addr);
5     int ret = i2c_endTransmission(&i2c_dev, false);
6     if (ret != I2C_ET_SUCCESS) {
7     DEBUG_TRACE("I2C eT=%d", ret);
8     return 0;
9     }
10    ret = i2c_requestFrom(&i2c_dev,I2C_HDC2010_ADDR, length, true);
11    if (ret != length) {
12    DEBUG_TRACE("I2C rF=%d", ret);
13    return 0;
14    }
15
16    length = i2c_available(&i2c_dev);
17    for (int i = 0; i < length; i++) {
18    data[i] = i2c_read(&i2c_dev);
19    }
20
21    return length;
22 }
```

The lis3dh_write API is to write the data for the lis3dh register.

```
1 static int lis2dh12_write(uint8_t reg_addr, uint8_t *data, int length)
2 {
3     i2c_beginTransmission(&i2c_dev,I2C_HDC2010_ADDR);
4     i2c_write_byte(&i2c_dev, reg_addr);
5     length = i2c_write_block(&i2c_dev, data, length);
6     int ret = i2c_endTransmission(&i2c_dev, true);
7     if (ret != I2C_ET_SUCCESS) {
8     DEBUG_TRACE("I2C eT=%d", ret);
9     return 0;
10    }
11    return length;
12 }
```

CONFIDENTIAL

## 4.3 SPI Driver

The ATM2/3 can support general purpose four pins SPI master port with mode 0 (CPOL=0, CPHA=0), and the SPI port clock frequency is programmable and ranges from 7.8 KHz to 8 MHz. The Atmosic application framework provides SPI modules with the associated APIs for SPI read/write communication.

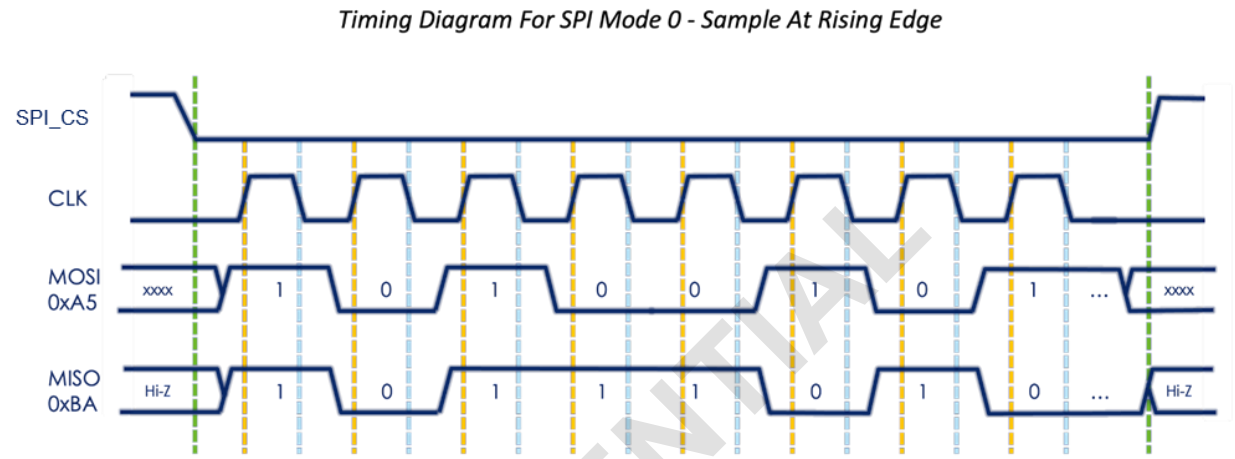



*Figure 24- Timing Diagram For SPI Mode 0 - Sample At Rising Edge*

### 4.3.1 Pin Assignment for SPI

For ATM2/3, it supports 2 set SPI and please refer to section 2.7 of the Pin multiplexing in ATM3/ATM2 Reference Manual for the related pin. There are four pins - SPI_CS, CLK, MISO, MOSI for SPI communication.

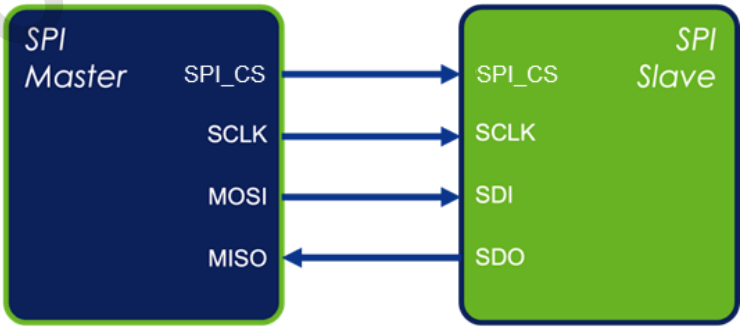



*Figure 25- SPI Pin*

Please make related pin assignment with PIN_SELECT API and enable it with WRPR_CTRL_SET.

CONFIDENTIAL

```
EX: SPI0, CS: P10, CLK: P20, MOSI: P22,
PIN_SELECT(13, SPI0_MISO);
PIN_SELECT(22, SPI0_MOSI);
PIN_SELECT(10, SPI0_CS);
PIN_SELECT(20, SPI0_CLK);

WRPR_CTRL_SET(CMSDK_SPI0, WRPR_CTRL__CLK_ENABLE);
```

### 4.3.2 SPI Module

The SPI module is implemented in rom code. The spi module exports the APIs for application layer to read/write with SPI slave device.

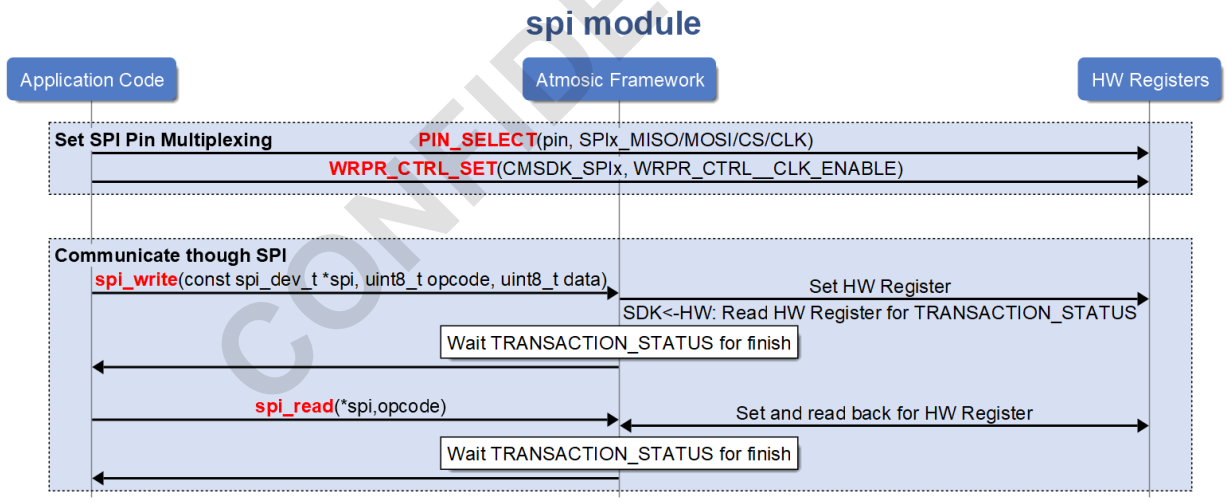



*Figure 26 - SPI Module*

The input spi_dev_t for the SPI APIs can be configured according to the SPI set and SPI clock. The CMSDK_AT_APB_SPI_TypeDef can be selected to CMSDK_SPI0 or CMSDK_SPI1. For SPI clock frequency, it can be set by the clkdiv. SPI clock would be system clock/(2*(clkdiv+1)), and the system clock is 16 MHz.

CONFIDENTIAL

```c
typedef struct spi_dev_s {
    CMSDK_AT_APB_SPI_TypeDef *base;
    uint16_t clkdiv;
    uint8_t dummy_cycles;
} spi_dev_t;

EX: SPI 0 with 4MHz clock frequency
static spi_dev_t const 4M_spi0 = { CMSDK_SPI0, 1, 0};
```

For SPI communication, it transits with 1 byte opcode. The opcode byte can not be removed. It would be the first byte data in MOSI, and the first byte data in MISO would be updated to the OPCODE_STATUS register.

```c
#define SPI_TRANSACTION_STATUS__OPCODE_STATUS__SHIFT                          8
#define SPI_TRANSACTION_STATUS__OPCODE_STATUS__WIDTH                          8
#define SPI_TRANSACTION_STATUS__OPCODE_STATUS__MASK                 0x0000ff00U
#define SPI_TRANSACTION_STATUS__OPCODE_STATUS__READ(src) \
                    (((uint32_t)(src)\
                    & 0x0000ff00U) >> 8)
```

For the SPI module, it provides read/write APIs for 1 to 4 bytes. See Table 17.

Table 17 - Atmosic SPI API Description

| API Name | Purpose | Parameter Description |
|---|---|---|
| uint8_t spi_read(const spi_dev_t *spi, uint8_t opcode)<br>uint16_t spi_read_2(const spi_dev_t *spi, uint8_t opcode)<br>uint32_t spi_read_3(const spi_dev_t *spi, uint8_t opcode)<br>uint32_t spi_read_4(const spi_dev_t *spi, uint8_t opcode) | Read 1~4 byte data | spi_dev_t : spi configuration<br>opcode: the first byte data for MOSI<br>return: the 2nd~5th byte data of MISO |
| void spi_write(const spi_dev_t *spi, uint8_t opcode, uint8_t data)<br>void spi_write_2(const spi_dev_t *spi, uint8_t opcode, uint16_t data)<br>void spi_write_3(const spi_dev_t *spi, uint8_t opcode, | Write 1~4 byte data to slave | spi_dev_t : spi configuration<br>opcode: the first byte data for MOSI<br>data: the 2nd~5th byte data for MOSI |

CONFIDENTIAL

| API Name | Purpose | Parameter Description |
|---|---|---|
| uint32_t data)<br>void<br>spi_write_4(const spi_dev_t *spi, uint8_t opcode,<br>uint32_t data) | | |

CONFIDENTIAL

# Revision History

| Date | Version | Description |
|---|---|---|
| September 26, 2022 | 0.90 | Updated Atmosic application framework module list for SDK 5.2.0. Renamed to Embedded Application Development Application Note. |
| May 13, 2022 | 0.80 | Added support for SDK 5.1 with ATM3330e/ATM3330, Table 1 - List of Supported Devices, Table 2 - Atmosic application framework module list, Table 3 - atm_gap_param_t Atmosic GAP parameters, Table 4 - Device Configurations Atmosic GAP Modules, sections Atmosic GAP Modules, Initialization, Atmosic Advertisement Modules, Figure 4 - Initialization Process, Figure 5 - Advertisement Modules, Figure 19 - Atmosic Application Framework Initialization Sequence. |
| September 29, 2021 | 0.70 | Renamed this document and added System Utilities and Hardware Drivers sections |
| July 20, 2021 | 0.60 | Updated various sections for SDK 4.1.0 |
| March 30, 2021 | 0.50 | Initial version created. |

CONFIDENTIAL

ATMOSIC TECHNOLOGIES – DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2020-2022 by Atmosic Technologies. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders.

CONFIDENTIAL

Atmosic™

Atmosic Technologies | 2105 S. Bascom Ave. | Campbell CA, 95008
**www.atmosic.com**